

Solving Nonstationary Non-Markovian Queues: A Transfer Learning Neural Network Approach

Spyros Garyfallos

Computer Architecture, Universitat Politècnica de Catalunya, Barcelona, Spain, spyridon.garyfallos@upc.edu
Amazon, Amazon Web Services (AWS), New York City, NY, USA, spyrosg@amazon.com

Yunan Liu

Amazon, Supply Chain Optimization Technology, New York City, NY, USA, yunanliu@amazon.com

Xianzhe Wang

Institute of Transportation Studies, University of California, Davis, CA, USA, xzhwang@ucdavis.edu

Pere Barlet-Ros, Alberto Cabellos-Aparicio

Computer Architecture, Universitat Politècnica de Catalunya, Barcelona, Spain, pere.barlet@upc.edu, alberto.cabellos@upc.edu

Abstract. Empirical studies show that real-world queueing systems—such as contact centers and healthcare facilities—exhibit non-Markovian characteristics, with non-exponential service and abandonment times, and nonstationary dynamics due to time-varying arrivals and staffing. These features make performance analysis and capacity planning highly challenging. Traditional methods like heavy-traffic approximations or simulation are often computationally intensive and lack accuracy for practical decision-making. This paper introduces QUANT (Queue Analysis via Neural-network Transfer learning), a neural network framework for analyzing and optimizing nonstationary non-Markovian queueing systems. Trained on synthetic data, QUANT captures the underlying spatio-temporal patterns of complex queues, and it incorporates transfer learning to adapt quickly to new environments with minimal additional data. QUANT offers major advantages: it reduces training cost by orders of magnitude compared to prior learning-based methods, delivers predictions that are significantly faster than traditional simulation, and maintains high accuracy across a range of performance metrics. The snapshot-based recurrent nature of QUANT allows for simple optimization algorithms for optimal staffing decisions under arbitrary objectives and constraints. By integrating accurate prediction with real-time optimization, QUANT transforms queueing models from passive analysis tools into active decision-support systems for dynamic capacity management in complex service operations.

Key words: non-Markovian queues; nonstationary queues; optimal staffing in queues; transfer learning

1. Introduction

The analysis of queueing systems with nonstationary demand and service capacity remains a central challenge in operations research and applied probability. Modern service operations—ranging from cloud computing platforms that must provision resources in real-time, to healthcare systems with fluctuating patient arrivals, to call centers with dynamic staffing schedules—are characterized by nonstationary customer demand and flexible, time-dependent capacity (Brown et al., 2005; Shi et al., 2014). These systems often exhibit non-Markovian dynamics, as service and abandonment

times need not follow exponential distributions, further complicating performance analysis (Brown et al., 2005). At the same time, practical decision-making requires models that can adapt quickly to evolving environments while balancing multiple, sometimes conflicting, performance objectives.

1.1. Challenges of Nonstationary Non-Markovian Queues

The analysis of *nonstationary non-Markovian* (NSNM) queueing systems presents formidable challenges that far exceed those encountered in stationary or Markovian settings. The presence of nonexponential service- and abandonment-time distributions breaks the Markov property, precluding the use of convenient state-transition formulations. At the same time, time-varying arrival rates and staffing functions eliminate the notion of equilibrium behavior, forcing the analysis to focus on transient system dynamics that evolve continuously over time. The combination of a non-Markovian probability structure and nonstationary model parameters therefore makes NSNM queues notoriously difficult to analyze, as both stochastic complexity and temporal variability must be captured simultaneously.

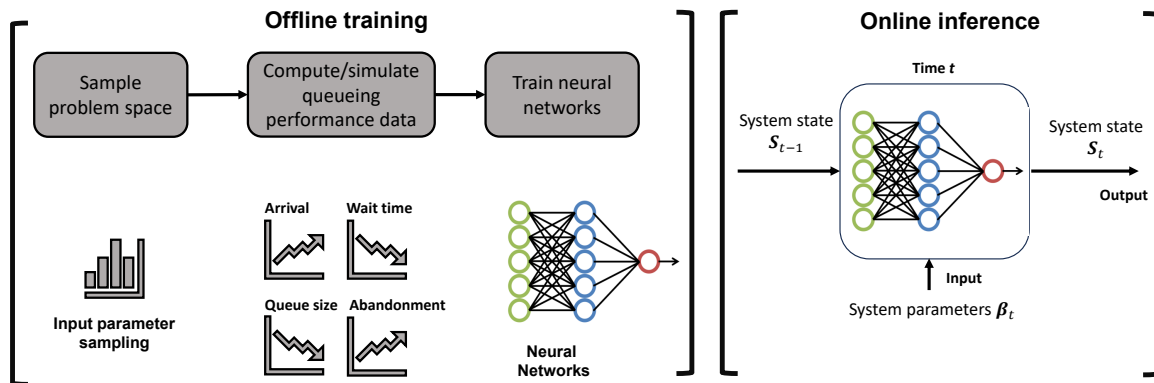
Heavy-Traffic Approximations. Heavy-traffic approximations are perhaps the most effective resort currently available for studying NSNM queueing systems. These asymptotic methods, including fluid and diffusion limits, remain among the few analytical tools capable of providing tractable insights when exact analysis is infeasible. Fluid limits describe the system's temporal evolution by averaging out stochastic variability (Liu and Whitt, 2012a; Whitt, 2006; Liu and Whitt, 2011, 2014a). This approach requires solving coupled differential equations that capture both time-varying demand and capacity, often resulting in highly intricate system dynamics. While fluid models provide useful approximations for mean performance, their deterministic nature prevents them from characterizing distributional measures such as the tail probability of the waiting time, $\mathbb{P}(W(t) > \tau)$ (Liu, 2018). Their accuracy further deteriorates in moderate- and low-traffic regimes or when the system frequently alternates between underloaded and overloaded states (Liu and Whitt, 2012a).

Diffusion limits, derived from the *functional central limit theorem* (FCLT), offer a stochastic refinement to fluid models but remain challenging to derive for general settings. They are most tractable in Markovian systems (Mandelbaum et al., 1998), whereas developing FCLT limits for queues with nonexponential service times and time-varying capacities is exceptionally difficult (Aras et al., 2018; Liu et al., 2016). Even when available, the resulting diffusion models often involve complex stochastic differential equations (Liu and Whitt, 2014a) and exhibit limited accuracy for systems of realistic, finite scale.

1.2. A Neural-Network Transfer-Learning Approach

To address the need for effective performance analysis and decision making in NSNM queues, we propose a machine learning-based framework for learning and optimizing their transient dynamics. In this paper, we present *Queue Analysis via Neural-network Transfer learning* (QUANT), a framework designed to address fundamental limitations in existing queueing analysis methods while introducing effective capabilities for service system capacity optimization.

Figure 1 The training data generation, model training, and online inference pipeline of our method.



Our QUANT framework is a queue-specialized neural network prediction model that recurrently predicts desired service-level metrics, such as queue length, waiting time, and the probability of delay, over time. Its development follows a three-stage pipeline that strategically leverages the queueing domain knowledge. The process begins with data generation, which capitalizes on the fact that a complex non-Markovian queue has a simpler Markovian counterpart: a system where all random times (e.g., service and abandonment) are made exponential, holding all other parameters (e.g., arrival rate, staffing level, and mean service and abandonment times) identical. The training data for this Markovian counterpart are computed efficiently by solving the underlying time-varying Markov chains, yielding a large, high-fidelity dataset. This is then supplemented by a much smaller simulated dataset for the actual non-Markovian system, having general service and abandonment distributions.

During model training, a transfer learning strategy is employed that mirrors this structural relationship: a base neural network is first pretrained on the large Markovian dataset to learn the fundamental dynamics dictated by characteristics of the queueing system other than the non-Markovian distributions (e.g., time-varying arrival rate and staffing). A secondary transformer module is then trained on the non-Markovian simulation data, learning only the performance corrections required to adapt the Markovian base predictions to the specific distributional shapes of

the target system. Finally, in the online inference stage, the fully trained model leverages its learned dynamics to enable real-time prediction of key performance metrics and support rapid optimization of staffing decisions under arbitrary service-level constraints. See Figure 1 for an illustration of the pipeline of QUANT.

1.3. A Motivating Example

We quickly illustrate QUANT's performance on several nonstationary queueing systems. We consider a nonstationary Markovian queueing system $M_t/M/n_t + M$ (with M denoting exponential distributions, and subscript t denoting time-varying arrival and staffing) and two non-Markovian variates: $M_t/H_2/n_t + H_2$ (hyperexponential service and abandonment) and $M_t/E_4/n_t + E_4$ (Erlang service and abandonment)¹. Both systems have identical means ($1/\mu = 1$ and $1/\theta = 1$) but represent inverse magnitudes of distributional dispersion, with squared coefficient of variation $SCV_{H_2} = 4$ and $SCV_{E_4} = 1/4$ respectively. Complete distributional specifications are provided in Section 4.2.

The base QUANT neural networks are first trained on the corresponding $M_t/M/n_t + M$ Markovian system with the same distributional means, then transferred to the two non-Markovian systems. QUANT demonstrates remarkable accuracy in predicting system performance across multiple metrics simultaneously, including probabilistic predictions such as *tail probability of delay* (TPoD).

Figure 2 shows QUANT's predictions closely follow ground truth across queue length $Q(t)$, mean waiting time, and TPoD $\mathbb{P}(W(t) > 0.5)$ throughout the operational period. The framework accurately captures dynamics during rapid transitions between underloaded and overloaded states, correctly predicting both the magnitude and timing of performance changes. Particularly noteworthy is QUANT's accurate prediction of tail probability metrics, which depend on complex interactions between arrival variability, service time distributions, and abandonment behavior. Additional numerical experiments in Section 3 evaluate QUANT's performance across diverse system configurations and transfer learning scenarios, and Section 5 demonstrates optimization applications.

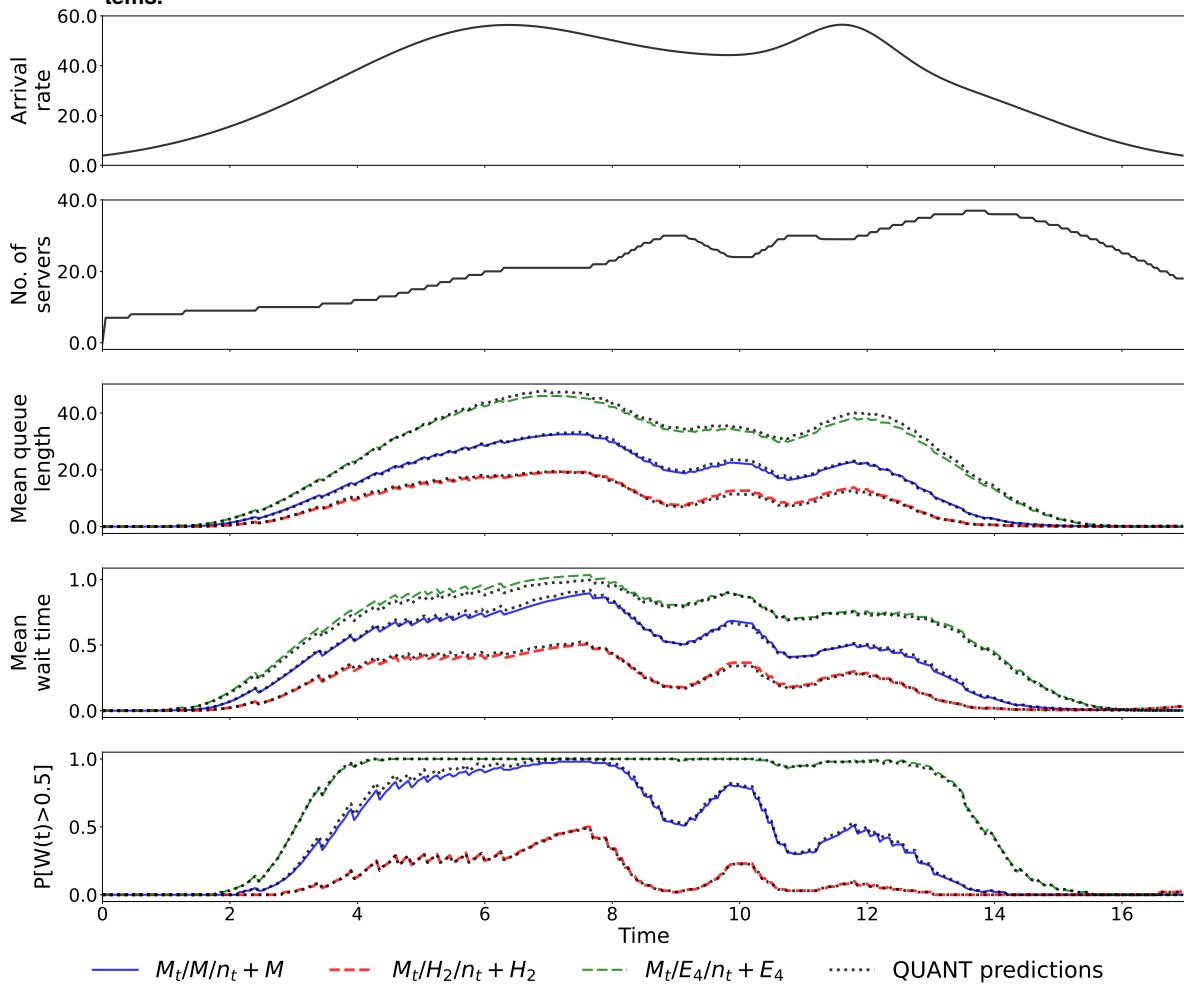
1.4. Contributions and Organization

We summarize the main contributions of this paper.

- **Deep learning for nonstationary non-Markovian queues.** We develop a deep learning framework, QUANT, for predicting transient performance metrics in NSNM queueing systems with time-varying arrival rates, staffing functions, and general random times (i.e., service and

¹ All three systems are special cases of the general $G_t/GI/n_t + GI$ type; the detailed definitions are introduced in Section 3

Figure 2 QUANT predictions (dotted lines) versus simulation ground truth for Markovian and non-Markovian systems.



abandonment). QUANT uses a recurrent neural network (RNN) architecture to compute key service-level metrics, including mean waiting time, mean queue length, abandonment probability, and tail delay probabilities, as functions of time.

- **New neural network architecture for time-varying staffing.** Unlike previous methods that rely on the pointwise steady state (PSS) approximation under constant staffing and therefore requiring customer abandonment to ensure a well-defined steady-state for arbitrary arrival and staffing combinations, we design a more advanced neural network structure that does not depend on PSS. This extension not only reduces the need for simulating the steady-state training data, but it also enables modeling a broader class of queueing systems, including $G_t/GI/n_t$ without customer abandonment.

- **Transfer learning to accelerate training.** We integrate transfer learning techniques into QUANT to construct prediction models for non-Markovian queues by leveraging models trained

on their Markovian counterparts. For the latter, we develop an efficient algorithm to compute exact service-level metrics, which serves as a performance oracle. This approach removes computational bottlenecks through more efficient data-generation strategies, substantially reducing the reliance on heavy simulation and reducing training data requirements by up to 99% when adapting to new distributional settings.

- **Efficient staffing optimization under service-level constraints.** In many real-world service systems such as call centers and data centers, decision makers often aim to satisfy desired constraints on multiple service-level metrics rather than a single one (e.g., waiting time, queue length, and tail delay probability). As an immediate application, we leverage QUANT to develop an efficient staffing algorithm that generates the optimal time-varying service capacity to meet multiple service-level constraints at designated targets.

Organization of the paper. In Section 2 we review the relevant literature. In Section 3 we introduce the QUANT framework; we explain the architecture design at the macroscopic, mesoscopic, and microscopic levels. In Section 4 we discuss our data generation strategy for model training, and we evaluate its performance. In Section 5 we present a staffing algorithm which uses QUANT for capacity planning adjustments subject to constraints on multiple service level metrics. Finally, we give concluding remarks in Section 6.

2. Related Literature

The present work builds upon and extends research in three primary domains: the performance analysis of NSNM queueing systems, time-varying staffing to stabilize performance in nonstationary queues, and the application of machine learning techniques to queueing theory. This section reviews the relevant literature in these areas.

2.1. Nonstationary Non-Markovian Queue: Performance Analysis and Capacity Planning

The analysis of queueing systems with nonstationary parameters, such as nonstationary arrival rates and time-varying staffing levels, has been an important area of queueing theory due to its practical relevance in real-world service systems like call centers and healthcare facilities. Common approaches for analyzing nonstationary queues often involve fluid and diffusion approximations. Mandelbaum et al. (1998) investigated many-server heavy-traffic limits for Markovian queueing networks with nonstationary parameters. Liu and Whitt (2012a, 2014a) developed fluid and FCLT limits for $G_t/GI/s_t + GI$ queues with nonexponential distributions and nonstationary arrivals. See Whitt (2018) for a broad overview of research on time-varying queues.

Our work is related to the literature on the staffing problems of queueing systems with time-varying arrivals. Pointwise stationary approximation (Green et al., 2007) has been proven useful in staffing systems with shorter service times, following the basic idea of approximating a nonstationary model at each time point by a stationary model. The modified-offered-load (MOL) approximation has been developed to design staffing functions to control performance functions, including the probability of delay (PoD), mean waiting time, and probability of abandonment (PoA) (Jennings et al., 1996; Liu and Whitt, 2012b, 2014b, 2017). Also see He et al. (2016); Sun and Liu (2021) for the extension to queues with non-Poisson arrival processes. Feldman et al. (2008) developed a simulation-based iterative staffing algorithm (ISA) to control the PoD. The idea of ISA was later extended by Defraeye and Van Nieuwenhuyse (2013); Konrad and Liu (2025a) to treat the TPoD. Unlike ISA, Liu (2018) was the first to develop an analytic staffing method for queues with exponential service times to stabilize the TPoD and prove the asymptotic correctness of the staffing algorithm. Liu et al. (2022) further extended results in Liu (2018) to queues having multiple customer classes, each having its own TPoD target. Distinct from the above literature, our present study conducts performance analysis and develops staffing functions for the more general NSNM queues. In addition, we aim to satisfy constraints involving multiple performance metrics.

2.2. Machine Learning in Queueing Systems

Recent developments in deep learning have fundamentally transformed the analysis and optimization of queueing systems, moving beyond traditional analytical approaches to embrace data-driven methodologies. Theoretical advances have established deep learning's capability to approximate complex queueing behaviors (Sherzer et al., 2022, 2025). Garyfallos et al. (2025) developed NeuraliNQ, a neural-network-based prediction method for non-Markovian queues under a constant staffing level. However, when applied to general NSNM queueing systems, NeuraliNQ faces three key limitations. First, its reliance on PSS leads to substantial simulation overhead. Second, this reliance restricts the method to systems with customer abandonment; without abandonment, steady-state distributions do not exist under overloaded conditions. Finally, NeuraliNQ cannot accommodate time-varying staffing functions, which prevents it from being used for staffing optimization.

QUANT extends these efforts by not only handling more general NSNM queueing systems but also by directly addressing the shortcomings of NeuraliNQ and related models. First, it advances beyond fixed-capacity settings by supporting time-varying staffing functions $n(t)$, a critical step for modeling real service environments where supply levels adjust dynamically to fluctuating demand. Second, by eliminating reliance on PSS, QUANT can model a broader class of systems, including

NSNM queues without customer abandonment. Third, the removal of PSS also reduces the need for costly steady-state simulations at each capacity level, yielding substantial computational savings and improving scalability. Fourth, because QUANT can rapidly predict the transient performance under arbitrary time-varying staffing functions, it enables direct integration with optimization routines to solve the optimal staffing problem—determining time-dependent capacity adjustments that satisfy service level constraints while minimizing operating costs.

3. Proposed Methodology

The QUANT framework is designed for the real-time performance analysis and optimization of general NSNM queueing systems. In particular, we consider the $G_t/GI/n_t + GI$ system which is characterized by a general arrival process with a nonstationary rate $\lambda(t)$ (the G_t), time-varying number of servers $n(t)$ (the n_t), *independent and identically distributed* (i.i.d.) service times following a general distribution G (the GI) with mean $1/\mu$ for $\mu > 0$, and customer abandonment according to i.i.d. random variables following a general distribution F (the $+GI$) with mean $1/\theta$ for $\theta \geq 0$. When $\theta = 0$, our model reduces to the $G_t/GI/n_t$ queue without customer abandonment. All customers are served under the *first-come, first-served* (FCFS) discipline. Our objective is to predict the transient trajectory of key *service level* (SL) metrics $\mathbf{S}_t$, such as the mean waiting time, mean queue length, and TPoD over a finite horizon $t \in [0, T]$. Let $Q(t)$ and $B(t)$ denote the numbers of customers in the waiting queue and in service, let $W(t)$ be the waiting time at time t .

3.1. Structural Design and Architecture of the QUANT Framework

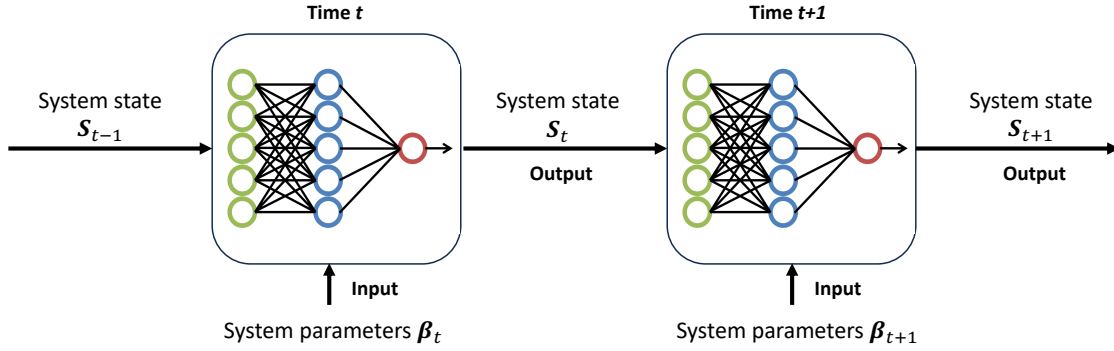
In the rest of this section, we describe the structural design of QUANT at three progressively finer levels of detail: *macroscopic*, *mesoscopic*, and *microscopic*.

3.1.1. Macroscopic level: At the *macroscopic* level, QUANT is a recurrent neural network that predicts the system state at the next time step based on the current time step. For each time t , QUANT serves as a function $\mathcal{F}$ that computes the system state $\mathbf{S}_{t+1}$ using (i) present state $\mathbf{S}_t$, and (ii) next-step model input β_{t+1} (e.g., $\lambda(t+1)$ and $n(t+1)$). Symbolically, the heart of QUANT is in the form of

$$\mathbf{S}_{t+1} = \mathcal{F}(\mathbf{S}_t, \beta_{t+1}), \quad t = 0, 1, 2, \dots, \quad (1)$$

where $\mathbf{S}_t$ is the system state representation at time t , and β_{t+1} encompasses the external model inputs for the interval $(t, t+1]$, such as the arrival rate $\lambda(t+1)$ and number of servers $n(t+1)$.

Figure 3 QUANT at the macroscopic level: A feedforward topology of time-indexed RNN.



State Representation: To accurately learn how a complex queueing system behaves, it is important to find a proper representation of its state S_t . While in NSNM models, detailed information like exact customer ages might theoretically be needed (Whitt, 2006), increasing input dimensionality can escalate model complexity and training effort. RNNs, by design, can learn to encode relevant history from a sequence of inputs using hidden state representations, but QUANT aims for efficiency by using a carefully selected snapshot state. Through experimentation, a parsimonious yet expressive system state representation was found to be:

$$S_t \equiv (\mathbb{E}[Q(t)], \mathbb{E}[B(t)], \mathbb{E}[W(t)]), \quad (2)$$

which tracks the mean queue length, mean number of busy servers, and mean waiting time at time t . This multi-faceted state captures information from both the waiting area (queue length, waiting time) and the serving area (server occupancy $\mathbb{E}[B(t)]/n(t)$). This is particularly important for systems with varying scales or those transitioning between underloaded and overloaded states, where different metrics dominate the system's characterization. The input vector β_{t+1} for predicting S_{t+1} includes the anticipated arrival rate $\lambda(t+1)$ and planned server capacity $n(t+1)$ for the next time step. We note here that the state representation can be expanded to incorporate additional performance metrics of the system, such as TPoD metrics, as we already demonstrated in Section 1.3.

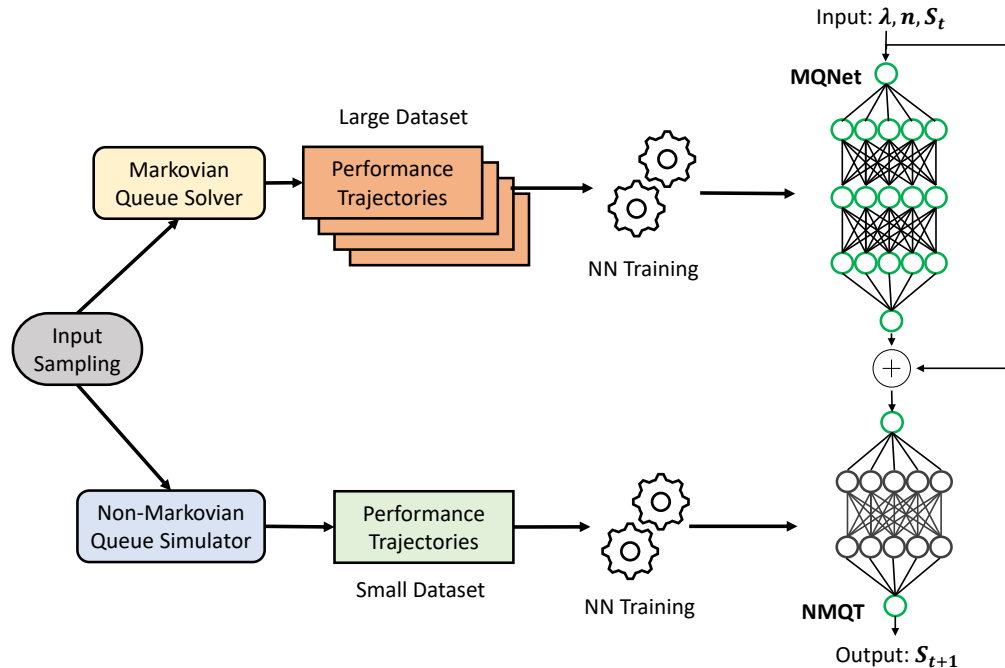
3.1.2. Mesoscopic level: Zooming into our one-step prediction module, we now explain QUANT's *mesoscopic*-level architecture. As illustrated in Figure 4, our one-step prediction model is comprised of two submodules: a *Markovian queue network* (MQNet) module and a *non-Markovian queue transformer* (NMQT) module.

Our key idea leverages transfer learning, a fundamental paradigm in deep neural networks that enables effective use of knowledge from source domains to improve learning in target domains

with limited data (Pan and Yang, 2010; Pratt et al., 1991; Caruana, 1997). We first pretrain a foundational MQNet module which aims to fully capture the queueing dynamics of the $M_t/M/n_t + M$ queueing system, having identical arrival rate $\lambda(t)$, staffing level $n(t)$, mean service time $1/\mu$, and mean abandonment time $1/\theta$ to the target NSNM $G_t/GI/n_t + GI$ system. This is the *Markovian counterpart* to the desired NSNM system. All performance distinctions of the two models are due to the distributional information of the stochastic elements (e.g., service and abandonment times) above their means in the NSNM queueing system, which is to be captured by the NMQT model.

While we experiment with different transfer learning approaches, such as fine-tuning the entire MQNet network or just its final layers, we present here the more interesting option of adding additional untrained layers of NMQT to the trained MQNet. As illustrated in Figure 4, MQNet takes inputs λ_{t+1} , n_{t+1} , and S_t , and NMQT takes the same inputs as MQNet, concatenated with MQNet's output, which is the Markovian queueing system's predicted S_{t+1} . Besides transfer learning, this option comes with the additional benefit of allowing the expansion of the dimensions of the problem space by adding additional input parameters to the NMQT network. One use case of particular interest with this transfer learning approach is the ability to add distributional parameters at that layer (e.g., the first k moments of the system's service and patience time distributions), generalizing the final NMQT network for a large family of queueing systems.

Figure 4 QUANT at the mesoscopic level: Transfer learning from Markovian queue to non-Markovian queue.



3.1.3. Microscopic level: At the microscopic level, MQNet employs a tandem structure with two specialized networks, $\mathcal{F}^Q$ and $\mathcal{F}^W$, that process count-based and time-based metrics separately:

$$\mathbf{S}_{t+1}^Q = \mathbf{S}_t^Q + \mathcal{F}^Q(\mathbf{S}_t, \beta_{t+1}) \quad (3)$$

$$\mathbf{S}_{t+1}^W = \mathcal{F}^W(\mathbf{S}_{t+1}^Q, \mathbf{S}_t^W, \beta_{t+1}), \quad (4)$$

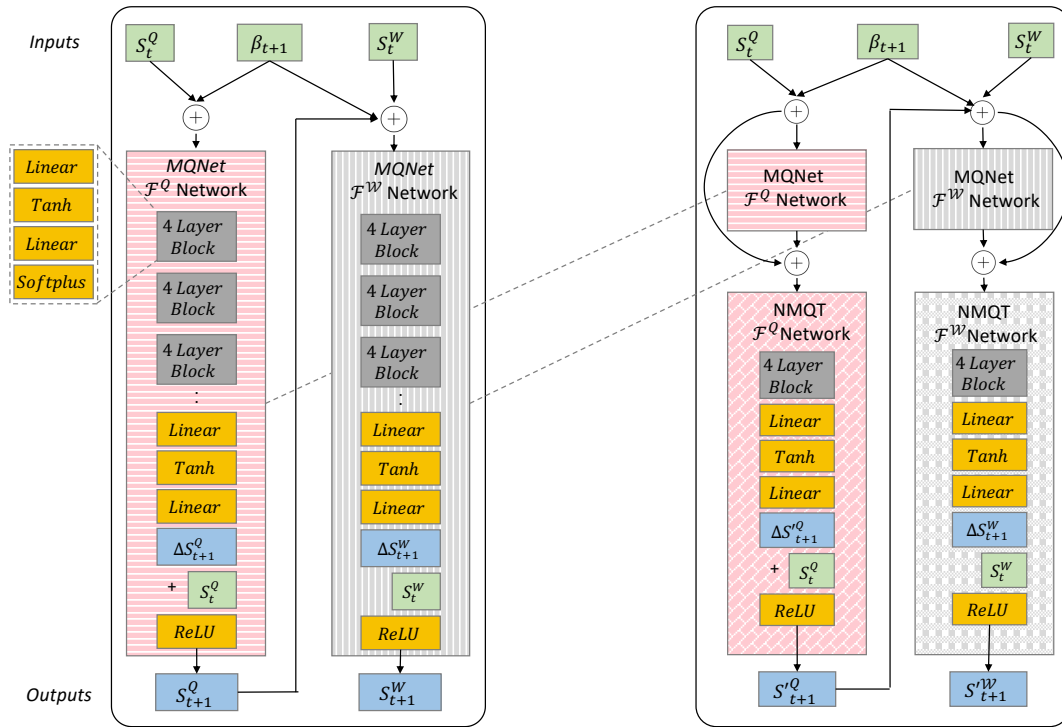
where $\mathbf{S}_t = (\mathbf{S}_t^Q, \mathbf{S}_t^W)$ with $\mathbf{S}_t^Q = (\mathbb{E}[Q(t)], \mathbb{E}[B(t)])$ and $\mathbf{S}_t^W = (\mathbb{E}[W(t)])$. The network $\mathcal{F}^Q$ predicts changes in queue-related counts, while $\mathcal{F}^W$ maps the predicted queue state to waiting time states. Symbolically, transfer learning changes (3) and (4) to:

$$\mathbf{S}_{t+1}^Q = \mathbf{S}_t^Q + \mathcal{F}_{NMQT}^Q(\mathbf{S}_t, \beta_{t+1}, \mathcal{F}_{MQNet}^Q(\mathbf{S}_t, \beta_{t+1})) \quad (5)$$

$$\mathbf{S}_{t+1}^W = \mathcal{F}_{NMQT}^W(\mathbf{S}_{t+1}^Q, \mathbf{S}_t^W, \beta_{t+1}, \mathcal{F}_{MQNet}^W(\mathbf{S}_{t+1}^Q, \mathbf{S}_t^W, \beta_{t+1})), \quad (6)$$

We provide complete details of the neural network architecture at the microscopic level. The networks $\mathcal{F}^Q$ and $\mathcal{F}^W$ consist of alternating linear layers with bias and nonlinearities.

Figure 5 QUANT at the microscopic level: MQNet (left) and NMQT (right).



Our NN architecture is designed to predict the *difference* of the state at time $t + 1$ compared to the state at t ($\Delta \mathbf{S}_{t+1}^{Q|W}$), and only at the final step do we compute $\mathbf{S}_{t+1}^Q$ by performing $\mathbf{S}_{t+1}^{Q|W} = \max(0, \Delta \mathbf{S}_{t+1}^{Q|W} + \mathbf{S}_{t+1}^{Q|W})$.

This NN architecture separates the count-based (e.g., $Q(t)$ and $B(t)$) metrics from the time-related metrics (e.g., $W(t)$ and $\mathbb{P}(W(t) > \tau)$), offering better convergence by grouping together similar spatiotemporal patterns. Second, the 4-layer repeating block with alternating Tanh and Softplus activations stabilizes gradient flow: Tanh provides zero-centered outputs that facilitate efficient gradient propagation, while Softplus maintains non-zero gradients everywhere, avoiding gradient vanishing. This architecture enables deeper networks with better learning capabilities and higher robustness. The final ReLU layer ensures positive results for the predicted metrics.

3.2. Data Generation and Model Training

A key innovation in QUANT lies in its training data generation strategy, which uses transfer learning. The model training, illustrated in Figure 1, involves the following steps. First, We develop an efficient algorithm, the *Markovian queue solver* (MQSolver), which serves as a performance oracle by computing exact performance metrics for Markovian queueing systems. Next, we design an efficient sampling strategy and use MQSolver to generate large-scale labeled training data used to supervise the training of the MQNet, which serves as a surrogate for the MQSolver model with rapid inference. Third, to generalize beyond the Markovian regime, we use stochastic simulation to generate a much smaller dataset for non-Markovian systems. Finally, we train a secondary neural network, NMQT, as a post-hoc correction module that learns the mapping from MQNet outputs to corrected predictions accounting for non-Markovian dynamics.

3.2.1. Step 1: Computing exact performance of Markovian queues. We develop MQSolver, a fast and scalable performance oracle which yields exact performance metrics for Markovian counterparts of target NSNM queueing systems. MQSolver solves the time-varying $M_t/M/n_t + M$ queueing model, which matches the target $G_t/GI/n_t + GI$ system in arrival rate $\lambda(t)$, staffing level $n(t)$, mean service time $\mathbb{E}[S] = 1/\mu$, and mean abandonment time $\mathbb{E}[A] = 1/\theta$, but uses exponential distributions.

MQSolver exploits the Markovian probability structure through uniformization techniques to efficiently compute exact performance metrics. The total queue $X(t)$ is modeled as a time-inhomogeneous birth-and-death process with birth rate $\lambda(t)$ and death rate $\mu(x \wedge n_t) + \theta(x - n_t)^+$ in state $x \geq 0$, where $x \wedge y \equiv \min(x, y)$ and $x^+ \equiv \max(x, 0)$. We compute the distribution $\pi_x(t) \equiv \mathbb{P}(X(t) = x)$ for all $t \in [0, T]$. Following Konrad and Liu (2025b), we compute the conditional waiting time $W(t)$ given $X(t) = x$. If $x < n_t$, we have $W(t) = 0$. If $x \geq n_t$, then all servers are busy and there are $q = x - n_t \geq 0$ customers waiting in the queue. Let $W_q(t)$ be the waiting time of a customer

arrival conditional on q waiting customers at t . We model $W_q(t)$ by a phase-type distribution, which is the time a pure death process reaches an absorbing state 0, beginning at an initial state $q + 1$. We compute the distribution of $W_q(t)$ following the discrete-time matrix geometric theory. Finally, we retrieve the unconditional waiting time distribution using the queue-length distribution as below:

$$\mathbb{P}(W(t) \leq \tau) = \sum_x \mathbb{P}(W(t) \leq \tau \mid X(t) = x) \pi_x(t) = \sum_x \mathbb{P}(W_q(t) \leq \tau) \pi_x(t), \quad \tau > 0, t \in [0, T] \quad (7)$$

Complete algorithmic details, including the uniformization technique for computing queue-length distributions and the phase-type approach for conditional waiting times, are provided in Appendix H.

3.2.2. Step 2: Sampling large-scale data from MQSolver to pretrain MQNet. We design a comprehensive data generation pipeline by systematically sampling diverse arrival-rate and staffing trajectories. The MQSolver developed in Step 1 is used to generate high-fidelity labels for these synthetic scenarios. With this large dataset, we train a foundational RNN model to approximate the dynamic input–output behavior of Markovian queues, serving as a high-capacity performance surrogate.

Generating Input Trajectories $\lambda(t)$ and $n(t)$: To pretrain MQNet, we require realistic and diverse nonstationary trajectories for arrival rates $\lambda(t)$ and service capacities $n(t)$. Let the operational parameter space be $\mathcal{B} = [\underline{\lambda}, \bar{\lambda}] \times [\underline{n}, \bar{n}]$ for some bounds $0 \leq \underline{\lambda} \leq \bar{\lambda} < \infty$ and $0 \leq \underline{n} \leq \bar{n} < \infty$. We generate temporal trajectories of these parameters using a time series composition of additive Gaussian processes (Bilancia et al., 2021). In particular, we define a basis function $B_k(t)$ as:

$$B_k(t) = \frac{C}{\max_{s \in [0, T]} P_k(s)} P_k(t), \quad \text{where} \quad P_k(t) = \sum_{i=1}^k c_i \psi_i(t) \quad \text{and} \quad \psi_i(t) = \frac{1}{\sigma_i \sqrt{2\pi}} e^{-\frac{(t-\mu_i)^2}{2\sigma_i^2}},$$

where $\psi_i(t)$ is the density function of a Gaussian distribution with mean μ_i and variance σ_i^2 , and C is a scaling factor to ensure the trajectories are within the predefined bounds $\mathcal{B}$. The parameters c_i, μ_i, σ_i for each Gaussian component are drawn from uniform distributions designed to generate diverse temporal profiles with varying peak locations, widths, and intensities. Complete parameter specifications are provided in Appendix B.

3.2.3. Step 3: Sampling small-scale non-Markovian queueing data via simulation. To capture real-world complexities such as general service and patience time distributions, we simulate the actual NSNM system under a much smaller set of scenarios of Step 2. These simulations produce a limited but high-quality dataset that reflects the non-Markovian characteristics of the queueing system.

3.2.4. Step 4: Transfer learning for training the non-Markovian transformer. We apply transfer learning to adapt the pre-trained MQNet to the target non-Markovian system using the small dataset from Step 3. This process consists of two stages: First,

- **Pre-training:** The complete MQNet architecture ($\mathcal{F}^Q$ and $\mathcal{F}^W$) is first trained on the large dataset generated by the MQSolver for the Markovian proxy model ($M_t/M/n_t + M$). This allows the network to learn the fundamental queueing dynamics driven by time-varying arrival rates $\lambda(t)$ and staffing levels $n(t)$.

- **Fine-tuning:** The pre-trained network is then fine-tuned using the small, targeted dataset from simulations of the actual $G_t/G/n_t + GI$ system. As outlined in Section 3.1.2, we implement the third transfer strategy: we freeze the pre-trained MQNet and train a new NMQT. NMQT takes the same inputs as MQNet, concatenated with MQNet’s output, and learns the specific corrections required to account for non-Markovian distributional shapes.

For fine-tuning with limited and potentially noisy simulation data, we use a significantly reduced learning rate compared to MQNet and larger batch sizes. This configuration helps mitigate overfitting and leads to stable, accurate predictions. This transfer learning strategy is central to QUANT’s efficiency. It leverages vast, cheaply generated analytical data to learn general behaviors and requires only a small amount of expensive simulation data to learn the specific corrections for non-Markovian characteristics.

4. Numerical Experiments

We now conduct extensive experiments to evaluate QUANT’s effectiveness and robustness. For effectiveness, we benchmark with the simulation result, referred to as the “ground truth”. We measure the prediction accuracy using the *time-averaged mean squared error* (TAMSE):

$$\mathcal{E}^\eta \equiv \frac{1}{T} \sum_{t=1}^T (e_t)^\eta, \quad e_t^\eta \equiv \hat{\eta}(t) - \eta(t), \quad (8)$$

where $\hat{\eta}(t)$ is the prediction of the SL metric $\eta(t)$ (e.g., mean waiting time and TPoD), and e_t^η is the prediction error at time step t . For robustness, we examine the QUANT performance across different system configurations, such as system scale, demand function, and various service and abandonment distributions.

4.1. Experimental Setup

Throughout, we hold mean service time $1/\mu = 1$ so we measure everything else in units of the mean service time. We consider a prediction period $[0, T]$ with $T = 17$, and a prediction time step $\Delta t = 0.05$.

We generate training data for the nonstationary arrival rate $\lambda(t)$ and staffing function $n(t)$ using time series compositions of Gaussian processes within operational bounds $\mathcal{B} = [\underline{\lambda}, \bar{\lambda}] \times [\underline{n}, \bar{n}]$, with $\underline{\lambda} = 0$, $\underline{n} = 1$, $\bar{\lambda} = \bar{n} = 100$. When simulating the ground truth, we use $N_s = 10,000$ independent Monte Carlo sample paths over $[0, T]$. For service and abandonment distributions we focus on various phase-type distributions (e.g., exponential, Erlang, and hyperexponential). Complete details on input sampling and simulation procedures are in Appendix B.

MQNet is trained using 10,000 training trajectories with 20% holdout for testing and contains roughly 174M total parameters across the $\mathcal{F}^Q$ and $\mathcal{F}^W$ networks. NMQT employs transfer learning with only 100 trajectories (1% of MQNet data) from non-Markovian simulations, using significantly reduced learning rates to preserve learned features. All models use L1 loss and the Adam optimizer. Full hyperparameter specifications and training procedures are in Appendix C.

4.2. Prediction Accuracy

We investigate our prediction accuracy across system scale and stochasticity using holdout test samples, with results summarized in Table 1. Accuracy is measured by TAMSE for four metrics: $\mathbb{E}[Q(t)]$, $\mathbb{E}[B(t)]$, $\mathbb{E}[W(t)]$, and $\mathbb{P}(W(t) > 1)$. For the base Markovian $M_t/M/n_t + M$ system, QUANT attains an average TAMSE of 0.07032 across these metrics.

System scale is characterized by the maximum staffing level $\hat{n} = \max_t n(t)$, with small ($\hat{n} \leq 41$), medium ($41 < \hat{n} \leq 72$), and large ($72 < \hat{n} \leq 100$) regimes. According to Table 1, QUANT maintains consistent accuracy across all scales.

We further test QUANT on NSNM queues with general service and abandonment distributions, including hyperexponential H_2 and Erlang (with 4 phases) E_4 cases. As shown in Table 1, prediction accuracy remains stable across these distributional settings, supporting the effectiveness of the transfer learning approach. Detailed distributional specifications and parameter settings are provided in the Appendix D.

Table 1 TAMSE accuracy across system configurations.

Category	Configuration	$\mathbb{E}[Q(t)]$	$\mathbb{E}[B(t)]$	$\mathbb{E}[W(t)]$	$\mathbb{P}(W(t) > 1)$
Distribution	$M_t/M/n_t + M$	0.095	0.083	0.062	0.041
	$M_t/H_2/n_t + H_2$	0.098	0.085	0.064	0.043
	$M_t/E_4/n_t + E_4$	0.097	0.084	0.063	0.042
System Scale	Small	0.118	0.103	0.078	0.046
	Medium	0.103	0.089	0.068	0.043
	Large	0.087	0.074	0.057	0.038

TAMSE values are reported separately for each performance metric.

These results demonstrate QUANT's robust performance across the various system configurations, maintaining the average TAMSE (across four metrics) below 0.1 across different system scales and distributional families. Also see Figure 2 for plots of the full prediction trajectories of three queueing systems, with service and abandonment times following exponential, hyperexponential, and Erlang distributions.

4.3. Runtime Efficiency

We compare the runtime performance of QUANT, MQSolver, and Monte-Carlo simulation for trajectories of duration $T = 17$ (corresponding to 340 time steps). All computations are performed on a single Intel Xeon E5-2686 v4 CPU core at 2.30 GHz and reported in CPU time. QUANT completes a full trajectory evaluation in approximately 0.35 seconds, which is substantially faster than MQSolver (about 60 seconds) and Monte-Carlo simulation (about 900 seconds).

Generating the training data requires approximately 7 CPU-days for Markovian systems and 1 CPU-day for non-Markovian transfer learning. For completeness, training the MQNet model takes about 30 minutes on a Tesla V100 SXM2 16GB GPU, while fine-tuning NMQT requires approximately 10 minutes.

4.4. Transfer Learning Data Efficiency

Table 2 Transfer learning performance versus fine-tuning dataset size.								
Dataset	100%	50%	25%	10%	5%	1%	0.5%	0.25%
Trajectories	10000	5000	2500	1000	500	100	50	25
$\mathbb{E}[Q(t)]$	0.096	0.097	0.097	0.098	0.098	0.101	0.121	0.200
$\mathbb{E}[B(t)]$	0.083	0.084	0.084	0.085	0.086	0.088	0.105	0.173
$\mathbb{E}[W(t)]$	0.063	0.063	0.064	0.064	0.065	0.066	0.079	0.130
$\mathbb{P}(W(t) > 1)$	0.041	0.042	0.042	0.043	0.043	0.044	0.052	0.086

An important practical consideration is the simulation data required for successful Markovian-to-non-Markovian transfer. We evaluate NMQT performance versus fine-tuning dataset size in the $M_t/H_2/n_t + H_2$ system. All TAMSE values are measured against Monte Carlo simulation ground truth and represent averages across the four predicted metrics: mean queue length $\mathbb{E}[Q(t)]$, mean busy servers $\mathbb{E}[B(t)]$, mean waiting time $\mathbb{E}[W(t)]$, and tail probability of delay $\mathbb{P}(W(t) > 1)$.

As shown in Table 2, the results reveal remarkable robustness down to approximately 1% (100 trajectories), where TAMSE remains at 0.0734 compared to 0.0703 for training with the full 10,000 non-Markovian trajectories—an accuracy degradation below 5%. Generating 100 trajectories requires approximately 1 CPU-day versus 100 CPU-days for full non-Markovian simulation.

Below this 1% threshold, accuracy degrades sharply, indicating a minimum limit for capturing distribution-specific corrections.

4.5. Summary

Experimental results establish four key findings: (1) QUANT achieves high prediction accuracy ($TAMSE < 0.1$) across diverse configurations, (2) computational speedup of 3-4 orders of magnitude versus simulation enables real-time analysis, (3) transfer learning requires only 100 simulation trajectories for effective adaptation, maintaining accuracy within 5% of full dataset performance, and (4) performance remains consistent across distributional families and system scales. These results validate QUANT as a practical tool for analyzing complex $G_t/G/n_t + GI$ systems, with transfer learning efficiency enabling rapid deployment in new environments, setting the stage for optimization applications in the following Section.

5. QUANT-based Optimal Staffing subject to Service Level Constraints

In this section, we leverage QUANT to determine optimal staffing functions for NSNM queues subject to constraints on designated service-level metrics. We consider the practically relevant case where we impose separate constraints on multiple service-level metrics simultaneously. The computational efficiency of QUANT's inference process enables iterative search methods to adjust capacity n_{t+1} at each time step until the predicted system state S_{t+1} meets the designated constraints. We call this approach *QUANT-based optimal staffing* (QUANT-OS).

5.1. The QUANT-OS Algorithm

QUANT-OS determines an optimal staffing sequence $\{n_t^*\}_{t=1}^T$ relative to a baseline staffing trajectory $\{n_t\}_{t=1}^T$, such that prescribed SL constraints are satisfied over the operational horizon $[0, T]$. At each decision epoch t , QUANT-OS evaluates a discrete set of candidate staffing levels and selects the smallest staffing level (i.e., the minimum number of servers) that satisfies all SL constraints. As illustrated in Figure 6, the key enabler of QUANT-OS is QUANT, whose extremely low inference cost permits real-time batch evaluation of a large number of candidate staffing configurations at each decision epoch.

One-Step Predictor and State Representation: Let $s_t \in \mathbb{R}^d$ denote the system state at time t , summarizing the queueing dynamics. At each time step, QUANT input is constructed as $\tilde{S}_t = [s_t, \lambda_{t+1}, n_{t+1}] \in \mathbb{R}^{1 \times d+2}$, where λ_{t+1} and n_{t+1} denotes the arrival rate and staffing level for the next period.

Algorithm 1 QUANT-OS: Adaptive Staffing via Batch Search

Require: Horizon T ; initial system state $\mathbf{s}_0 \in \mathbb{R}^d$; arrival-rate trajectory $\{\lambda_t\}_{t=1}^T$; baseline staffing $\{n_t\}_{t=1}^T$; QUANT one-step predictor $\mathcal{F}$; constraint functional Π (feasible iff $\Pi(\cdot) \leq 0$); staffing bounds $[\underline{n}, \bar{n}]$

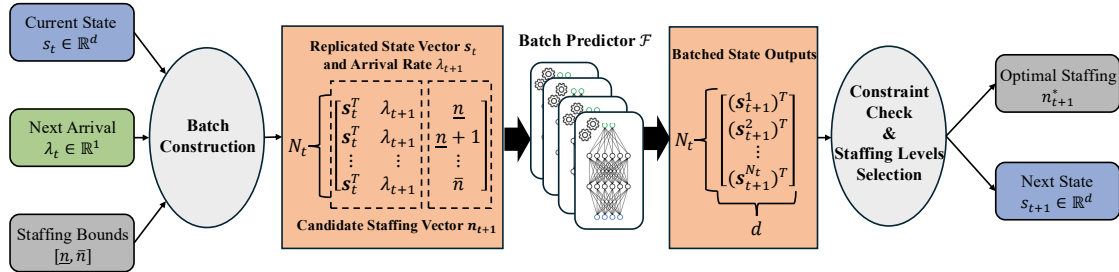
Ensure: Optimal staffing trajectory $\{n_t^*\}_{t=1}^T$

```

1: for  $t = 0$  to  $T - 1$  do
2:    $\tilde{\mathbf{S}}_t \leftarrow [\mathbf{s}_t^\top \ \lambda_{t+1} \ n_{t+1}] \in \mathbb{R}^{1 \times (d+2)}$ 
3:    $\tilde{\mathbf{S}}_{t+1}^o \leftarrow \mathcal{F}(\tilde{\mathbf{S}}_t), \ \tilde{\mathbf{S}}_{t+1}^o \in \mathbb{R}^{1 \times d}$ 
4:   if  $\Pi(\tilde{\mathbf{S}}_{t+1}^o) \leq \mathbf{0}$  then
5:      $n_{t+1}^* \leftarrow n_{t+1}$  ▷ Baseline is already feasible
6:   else
7:      $\mathcal{N}_t \leftarrow \{\underline{n}, \underline{n} + 1, \dots, \bar{n}\}, \ N_t \leftarrow |\mathcal{N}_t| = \bar{n} - \underline{n} + 1, \ n^{(i)} \in \mathcal{N}_t.$ 
8:      $\lambda_{t+1} \leftarrow \lambda_{t+1} \mathbf{1}_{N_t} \in \mathbb{R}^{N_t}, \ \mathbf{S}_t \leftarrow \mathbf{1}_{N_t} \mathbf{s}_t^\top \in \mathbb{R}^{N_t \times d}, \ \mathbf{n}_{t+1} \leftarrow [n^{(1)} \ \dots \ n^{(N_t)}]^\top \in \mathbb{R}^{N_t}.$ 
9:      $\tilde{\mathbf{S}}_t \leftarrow [\mathbf{S}_t \ \lambda_{t+1} \ \mathbf{n}_{t+1}] \in \mathbb{R}^{N_t \times (d+2)}.$ 
10:     $\mathbf{S}_{t+1}^o \leftarrow \mathcal{F}(\tilde{\mathbf{S}}_t), \ \mathbf{S}_{t+1}^o \in \mathbb{R}^{N_t \times d}$  ▷ Evaluate one-step predictions in batch across all  $n \in \mathcal{N}_t$ 
11:     $\Phi_t \leftarrow \{(\mathbf{n}_t)_i \mid \Pi((\mathbf{S}_{t+1}^o)_i) \leq \mathbf{0}, i \in \{1, \dots, N_t\}\}.$  ▷  $(\mathbf{S}_{t+1}^o)_i, (\mathbf{n}_{t+1})_i$  denotes the  $i$ -th row of  $\mathbf{S}_{t+1}^o$  and  $\mathbf{n}_{t+1}$ 
12:    if  $\Phi_t \neq \emptyset$  then
13:       $n_{t+1}^* \leftarrow \min \Phi_t$ 
14:    else
15:       $n_{t+1}^* \leftarrow \bar{n}$ 
16:    end if
17:  end if
18:   $\tilde{\mathbf{S}}_t^* \leftarrow [\mathbf{s}_t^\top \ \lambda_{t+1} \ n_{t+1}^*] \in \mathbb{R}^{1 \times (d+2)}$ 
19:   $\mathbf{S}_{t+1}^* \leftarrow \mathcal{F}(\tilde{\mathbf{S}}_t^*), \ \mathbf{S}_{t+1}^* \in \mathbb{R}^{1 \times d}$  ▷ Use the optimal staffing to compute the initial state for period  $t+1$ 
20:   $\mathbf{s}_{t+1} = (\mathbf{S}_{t+1}^*)^\top \in \mathbb{R}^d.$ 
21: end for
22: return  $\{n_t^*\}_{t=1}^T$ 

```

Figure 6 Overview of the QUANT-OS algorithm and its batch evaluation mechanism.



Let B denote the batch size, i.e., the number of input states evaluated simultaneously. The QUANT predictor is a vectorized mapping $\tilde{\mathbf{S}}_{t+1}^o = \mathcal{F}(\tilde{\mathbf{S}}_t) : \mathbb{R}^{B \times (d+2)} \rightarrow \mathbb{R}^{B \times d}$, which maps each augmented

input state to a d -dimensional next period system state, and is designed to exploit modern neural network architectures that support efficient batch inference, thereby producing one-step-ahead state predictions for all B inputs simultaneously in a single forward pass.

Constraint Policy Function: Suppose there are k service-level (SL) metrics $\eta_1(t), \dots, \eta_k(t)$ with corresponding upper bounds $\bar{\eta}_1, \dots, \bar{\eta}_k$. We require that $\eta_i(t) \leq \bar{\eta}_i$ $i = 1, \dots, k$, $t \in [0, T]$. For instance, typical constraints include bounds on the mean waiting time $\mathbb{E}[W(t)]$, the mean queue length $\mathbb{E}[Q(t)]$, and the tail delay probability $\mathbb{P}(W(t) > w)$.

Let $\Pi(\tilde{\mathbf{S}}_{t+1}^o)$ denote a constraint evaluation function that maps the predicted next-period system state to a scalar measure of constraint violation. Specifically, we define $\Pi(\tilde{\mathbf{S}}_{t+1}^o) = h(\eta_1(t+1) - \bar{\eta}_1, \dots, \eta_k(t+1) - \bar{\eta}_k)$, where $h(\cdot)$ is monotone increasing in each argument. Although the formulation allows for a general aggregation function, throughout the remainder of this paper we focus on the special case $h(x_1, \dots, x_k) = \max\{x_1, \dots, x_k\}$, which captures the maximum violation across all SL metrics.

Batch Evaluation of Candidate Staffing Levels. If the baseline staffing n_{t+1} is feasible, QUANT-OS retains it. Otherwise, a candidate set $\mathcal{N}_t = \{\underline{n}, \underline{n} + 1, \dots, \bar{n}\}$ is constructed. For batch evaluation, the current state $\mathbf{s}_t$, arrival rate λ_t , and staffing level n_t are replicated across all candidates:

$$\mathbf{S}_t = \mathbf{1}_{|\mathcal{N}_t|} \mathbf{s}_t^\top \in \mathbb{R}^{|\mathcal{N}_t| \times d}, \boldsymbol{\lambda}_{t+1} = \lambda_{t+1} \mathbf{1}_{|\mathcal{N}_t|}, \mathbf{n}_{t+1} = [n^{(1)}, \dots, n^{(|\mathcal{N}_t|)}]^\top.$$

The stacked predictor input is $\tilde{\mathbf{S}}_t = [\mathbf{S}_t, \boldsymbol{\lambda}_{t+1}, \mathbf{n}_{t+1}] \in \mathbb{R}^{|\mathcal{N}_t| \times (d+2)}$. And a single tensorized forward pass yields $\mathbf{S}_{t+1}^o = \mathcal{F}(\tilde{\mathbf{S}}_t) \in \mathbb{R}^{|\mathcal{N}_t| \times d}$, which contains the predicted next-period states for all candidate staffing levels simultaneously.

Staffing Selection Rule. Let $(\mathbf{S}_{t+1}^o)_i$ denote the i th row of $\mathbf{S}_{t+1}^o$. The feasible staffing set is defined as $\Phi_t = \{n^{(i)} \in \mathcal{N}_t \mid \Pi((\mathbf{S}_{t+1}^o)_i) \leq 0\}$. The staffing decision is then

$$n_{t+1}^* = \begin{cases} \min \Phi_t, & \text{if } \Phi_t \neq \emptyset, \\ \bar{n}, & \text{otherwise.} \end{cases}$$

Validation Methodology: To validate QUANT-OS effectiveness, we employ a two-stage verification process. First, we run QUANT-OS given an arrival pattern λ_{t+1} , an original staffing function n_{t+1} and a constraint policy $\Pi(\cdot)$, to obtain an optimized staffing function n_{t+1}^* and QUANT's predicted performance. Second, we simulate the actual $G_t/G/n_t + GI$ system using n_{t+1}^* and we measure the simulated satisfaction of the constraint policy. We declare constraint satisfaction when the simulated metric remains within bounds for at least 95% of the operational horizon.

5.2. Experimental Configuration

We construct 200 distinct operational scenarios drawn from the experimental configuration of Section 3, including the three different distributional configurations presented there. We evaluate the performance of QUANT-OS under three classes of time dependent service level constraints: mean waiting time $\mathbb{E}[W(t)] \leq \bar{w}$, mean queue length $\mathbb{E}[Q(t)] \leq \bar{q}$, and TPoD constraints $\mathbb{P}(W(t) > w) \leq \alpha$. Complete parameter specifications are in Appendix E.

5.3. Convergence and Computational Performance

The QUANT-OS demonstrates robust and reliable convergence behavior throughout the evaluation. Across all tested instances, the method attains a 100% convergence success rate. Leveraging its batch evaluation structure, QUANT-OS completes each optimization run in sub-second time (0.8 seconds), resulting in negligible computational overhead even over long planning horizons. For completeness, we have also implemented two alternative optimization strategies based on binary search and gradient based updates. Their methodological details and comparative results are reported in the Appendix F.

All optimization methods considered benefit from QUANT's fast inference time, which requires approximately 1 millisecond per forward pass at each time step. This efficiency enables the use of multiple optimization strategies without incurring prohibitive computational cost.

5.4. Validation Against Simulation

Table 3 compares QUANT's predicted constraint satisfaction with simulation outcomes under staffing policies optimized by QUANT-OS. Simulation success rate is defined as the fraction of time periods in which simulated performance satisfies the specified constraints. For each scenario, baseline simulation under a staffing function $\{n_t\}_{t=1}^T$ is used to calibrate constraint thresholds as fixed percentages of the maximum observed performance, with mean-based thresholds set to $p \in \{25\%, 50\%, 75\%\}$ of $\max_t \mathbb{E}[W(t)]$ and $\max_t \mathbb{E}[Q(t)]$, and tail-based TPoD constraints specified by $w \in \{0.5, 1.0\}$ and $\alpha \in \{0.25, 0.50, 0.75\}$. In multi-constraint experiments, two service-level constraints are enforced simultaneously. Specifically, we combine a mean queue length constraint with a TPoD constraint, resulting in joint feasibility requirements. A staffing policy is considered feasible only if both constraints are satisfied at a given time t .

While constraints are enforced deterministically at the prediction level, simulation results achieve satisfaction rates between 94.0% and 98.0% across all tested configurations, demonstrating that QUANT-OS produces staffing policies that are sufficiently accurate for reliable capacity planning.

Table 3 Prediction accuracy and constraint satisfaction in simulation.

Constraint Type	Prediction Error (TAMSE)	Simulation Success Rate
$\mathbb{E}[W(t)] \leq \bar{w}$	0.095	97.0%
$\mathbb{E}[Q(t)] \leq \bar{q}$	0.088	97.5%
$\mathbb{P}(W(t) > w) \leq \alpha$	0.131	95.0%
Multi-constraint	0.140	94.0%

Simulation success rate represents percentage of time steps where simulated performance satisfies the threshold.

The majority of the violations happen in TPoD constraints, due to their increased sensitivity to very small staffing adjustments. Tighter constraints naturally show slightly higher violation rates, but even for the most stringent constraints tested, QUANT-OS achieves over 94% simulated satisfaction.

5.5. Robustness Across System Configurations

Table 4 examines QUANT-OS performance across different distributional families and arrival scales, following the same scale classification done for Table 1.

Table 4 Robustness analysis across system configurations.

Category	Configuration	Simulation Success Rate	Prediction Error (TAMSE)
Distribution	$M_t / M / n_t + M$	98.5%	0.121
	$M_t / E_4 / n_t + E_4$	97.0%	0.224
	$M_t / H_2 / n_t + H_2$	95.5%	0.360
System Scale	Small	96.5%	0.156
	Medium	97.0%	0.140
	Large	98.5%	0.132

Results aggregated across constraint types within each configuration category.

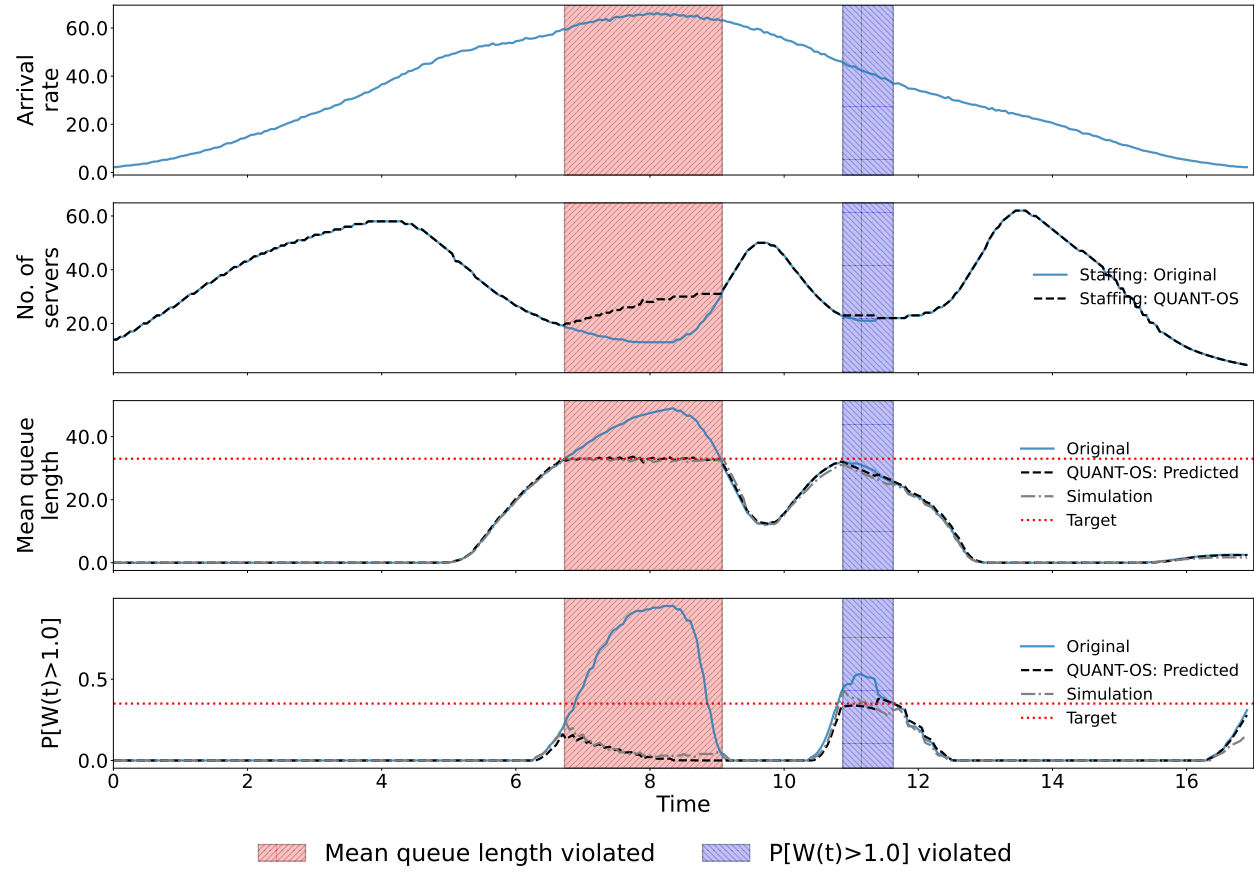
QUANT-OS maintains simulation success rate above 95% for all configurations tested. The algorithm performs optimally for Markovian systems, with modest degradation for high-variability hyperexponential systems. Arrival scale has minimal impact, with increasing accuracy as the systems scale up. This can be attributed to higher degrees of training data noise at low scales.

5.6. A Multi-Constraint Optimization Example

Practitioners across domains encounter SL agreements (SLAs) involving multiple service-level constraints. For example, cloud data centers manage request queues under tail-latency guarantees (Dean and Barroso, 2013), hospitals optimize staffing subject to waiting-time and capacity constraints (Bittencourt et al., 2018), and contact centers operate under joint constraints on TPoD and average waiting time (Koole and Mandelbaum, 2002).

In Figure 7 we demonstrate the QUANT-OS performance under two SL constraints: $\mathbb{E}[Q(t)] \leq \bar{q} = 33$ and $\mathbb{P}(W(t) > 1) \leq \tau = 0.35$ for all $t \in [0, T]$. The scenario features an arrival pattern with a peak of 70 customers per time unit at $t = 8$, and a suboptimal original staffing function.

Figure 7 QUANT-OS subject to multiple SL targets: performance comparison under original staffing plan (solid line) and optimized staffing plan (dashed line). There are two SL limits (dotted lines): queue length and TPoD.



Service and abandonment times follow hyperexponential H_2 distributions with mean 1 and SCV = 4, representing high variability. Simulation validation confirms effective performance: despite high-variability distributions and tight dual constraints, QUANT-OS achieves 96.7% constraint satisfaction across optimized steps (61 of 63).

6. Conclusions

This paper introduced QUANT, a framework that integrates performance prediction with real-time capacity optimization for nonstationary non-Markovian queueing systems with time-varying capacity. The framework achieves three primary contributions: eliminating PSS dependency, broadening applicability while reducing computational overhead by orders of magnitude, employing transfer learning that reduces simulation data requirements by up to 99%, and providing integrated optimization capabilities that achieve over 94% constraint satisfaction. Experimental results demonstrate that QUANT maintains time-averaged mean squared errors below 0.1 and computational speedups

of 3-4 orders of magnitude compared to simulation. QUANT’s principal contribution is transforming neural network-based queueing analysis from a specialized prediction tool into an integrated platform for operational decision-making.

Limitations and future Directions. The current framework has two main limitations. First, while QUANT’s transfer learning substantially reduces data requirements, the framework still lacks generalization across entire families of distributions—each new system configuration still requires targeted simulation data for fine-tuning. Second, the optimization methods assume static constraint policies and do not adapt to real-time system feedback.

A natural future extension would incorporate distributional parameters directly as neural network inputs, enabling a single model to generalize across entire distribution families. Extending QUANT to fully time-varying systems — $G_t/G_t/n_t + G_t$ where service and abandonment distributions vary temporally — would better capture real-world operations where service rates fluctuate due to worker fatigue, time-of-day effects, or evolving task complexity. Incorporating complex routing disciplines, including priority classes and skill-based routing where servers possess heterogeneous capabilities, would enhance practical applicability.

Extending QUANT to general queueing network topologies represents another natural progression. Graph Neural Networks provide a principled approach for capturing network structure and inter-queue dependencies, enabling the framework to handle arbitrary topologies and support both performance prediction and topological optimization of network structures, routing policies, and capacity allocations.

Current optimization methods determine staffing functions based on predicted demand and fixed constraint policies. Potential extensions include time-varying constraints adapting to operational priorities, feedback-adaptive policies adjusting staffing based on observed deviations, and reinforcement learning approaches using QUANT as an environment model. Multi-objective time-varying optimization frameworks could explicitly trade off competing goals, enabling more nuanced decision-making than simple constraint satisfaction.

QUANT represents a significant advance toward more powerful and tractable methods for analyzing and optimizing complex queueing systems. We envision QUANT enabling adaptive service systems that continuously optimize performance in response to changing conditions, improving service quality while reducing costs across diverse industries—from healthcare facilities to cloud computing platforms to contact centers. As service operations become increasingly complex and data-driven, frameworks like QUANT that combine domain knowledge with modern machine learning will play a crucial role in bridging theory and practice.

References

- Aras AK, Chen X, Liu Y (2018) Many-server gaussian limits for non-markovian queues with customer abandonment. *Queueing Systems* 89(1):81–125.
- Bilancia M, Manca F, Sansaro G (2021) *A Time Series Decomposition Algorithm Based on Gaussian Processes*, 577–592 (Springer International Publishing).
- Bittencourt O, Verter V, Yalovsky M (2018) Hospital capacity management based on the queueing theory. *International Journal of Productivity and Performance Management* 67(2):224–238.
- Brown L, Gans N, Mandelbaum A, Sakov A, Shen H, Zeltyn S, Zhao L (2005) Statistical analysis of a telephone call center: A queueing-science perspective. *Journal of the American Statistical Association* 100(469):36–50.
- Caruana R (1997) Multitask learning. *Machine learning* 28(1):41–75.
- Dean J, Barroso LA (2013) The tail at scale. *Commun. ACM* 56(2):74–80.
- Defraeye M, Van Nieuwenhuysse I (2013) Controlling excessive waiting times in small service systems with time-varying demand: An extension of the isa algorithm. *Decision Support Systems* 54(4):1558–1567.
- Feldman Z, Mandelbaum A, Massey WA, Whitt W (2008) Staffing of time-varying queues to achieve time-stable performance. *Management Science* 54(2):324–338.
- Garyfallos S, Liu Y, Barlet-Ros P, Cabellos-Aparicio A (2025) NeuraliNQ: a neural network method for the transient performance analysis in non-markovian queues. *Queueing Systems* 109(4), URL <http://dx.doi.org/10.1007/s11134-025-09952-3>.
- Green L, Kolesar P, Whitt W (2007) Coping with time-varying demand when setting staffing requirements for a service system. *Production Oper. Management* 16(1):13–39.
- He B, Liu Y, Whitt W (2016) Staffing a service system with non-Poisson nonstationary arrivals. *Probab. Engrg. Inform. Sci.* 30(4):593–621.
- Jennings OB, Mandelbaum A, Massey WA, Whitt W (1996) Server staffing to meet time-varying demand. *Management Science* 42(10):1383–1394.
- Konrad K, Liu Y (2025a) Achieving stable service-level targets in time-varying service systems: A simulation-based offline learning staffing algorithm. *2025 Winter Simulation Conference (WSC)*, 327–338.
- Konrad K, Liu Y (2025b) Real-time estimation for the waiting time distributions in time-varying queues. *2025 Winter Simulation Conference (WSC)*, 315–326.
- Koole G, Mandelbaum A (2002) Queueing models of call centers: An introduction. *Annals of Operations Research* 113(1–4):41–59.
- Liu Y (2018) Staffing to stabilize the tail probability of delay in service systems with time-varying demand. *Operations Research* 66(2):514–534.
- Liu Y, Sun X, Hovey K (2022) Scheduling to differentiate service in a multiclass service system. *Operations Research* 70(1):527–544.

- Liu Y, Whitt W (2011) A network of time-varying many-server fluid queues with customer abandonment. *Operations Research* 59(4):835–846.
- Liu Y, Whitt W (2012a) The $G_t/GI/s_t + GI$ many-server fluid queue. *Queueing Systems* 71(4):405–444.
- Liu Y, Whitt W (2012b) Stabilizing customer abandonment in many-server queues with time-varying arrivals. *Operations Research* 60:1551–1564.
- Liu Y, Whitt W (2014a) A many-server heavy-traffic limit for queues with time-varying parameters. *The Annals of Applied Probability* 24(1):61–126.
- Liu Y, Whitt W (2014b) Stabilizing performance in network of queues with time-varying arrival rates. *Probab. Engrg. Inform. Sci.* 28(4):419–449.
- Liu Y, Whitt W (2017) Stabilizing performance in many-server queues with time-varying arrivals and customer feedback. *Eur. J. Oper. Res.* 256(2):473–486.
- Liu Y, Whitt W, Yu Y (2016) Approximations for heavily-loaded $G/GI/n + GI$ queues. *Naval Research Logistics* 63:187–217.
- Mandelbaum A, Massey WA, Reiman MI (1998) Strong approximations for Markovian service networks. *Queueing systems* 30(1-2):149–201.
- Pan SJ, Yang Q (2010) A survey on transfer learning. *IEEE Transactions on knowledge and data engineering* 22(10):1345–1359.
- Pratt LY, Mostow J, Kamm CA (1991) Direct transfer of learned information among neural networks. *Proceedings of the ninth national conference on artificial intelligence* 584–589.
- Sherzer E, Baron O, Krass D, Resheff Y (2025) Approximating $G(t)/GI/1$ queues with deep learning. *European Journal of Operational Research* 322(3):889–907.
- Sherzer E, Senderovich A, Baron O, Krass D (2022) Can machines solve general queueing systems? *ACM SIGMETRICS Performance Evaluation Review*, volume 50, 85–86.
- Shi P, Chou M, Dai JG, Ding D, Sim J (2014) Models and insights for hospital inpatient operations: Time-dependent boarding time. *Management Sci.* 62(1):1–28.
- Sun X, Liu Y (2021) Staffing many-server queues with autoregressive inputs. *Naval Research Logistics* 68(3):312–326.
- Whitt W (2006) A multi-class fluid model for a contact center with skill-based routing. *Aeu-international Journal of Electronics and Communications* 60:95–102.
- Whitt W (2018) Time-varying queues. *Queueing models and service management* 1(2):79–164.

APPENDIX

In this appendix, we provide additional materials to supplement the main paper. Section A lists the nomenclature used throughout the paper. Section B describes the data generation procedures. Section C provides training procedures and hyperparameters. Section E details the optimization experimental configuration. Section F provides QUANT-OS implementation details including alternative optimization methods. Section G analyzes queues without customer abandonment. Section H provides complete algorithmic details for MQSolver.

Appendix A: Nomenclature

Table 5 summarizes the acronyms and terminology used throughout the paper.

Category	Acronym	Description
Problem setting	NSNM	Nonstationary non-Markovian
Proposed in this work	QUANT	QQueue Analysis via Neural-network Transfer learning
	MQNet	Markovian Queue Network
	NMQT	non-Markovian Queue Transformer
	QUANT-OS	QUANT-based Optimal Staffing
	MQSolver	Markovian Queue Solver
Metrics	SL	Service Level
	SLAs	Service Level Agreements
	TAMSE	Time-Averaged Mean Squared Error
	TPoD	Tail Probability of Delay
	PoD	Probability of Delay
	PoA	Probability of Abandonment
	PWT	Potential Waiting Time
	AWT	Actual Waiting Time
Prior methods	NeuraliNQ	Neural-network-based prediction method for non-Markovian queue
	PSS	Pointwise Steady State
	MOL	Modified-offered-load
	ISA	Simulation-Based Iterative Staffing Algorithm
Queueing primitives	FCFS	First-come, First-Served
Theory	FCLT	Functional Central Limit Theorem
General ML components	RNN	Recurrent Neural Network

Table 5 Summary of acronyms and terminology

Appendix B: Data Generation Procedures

This appendix provides complete details of the data generation procedures described in Section 3.2.2 and Section 3.2.3.

B.1. Input Sampling Strategy

To generate our training dataset, we use a basis function $B_k(t)$ as defined in Section 3.2.2.

Throughout the experiments presented in this paper, we set $k = 10$ parametric Gaussian components. The parameters of each i component are drawn uniformly from the following distributions: for $\lambda(t)$, the means μ_i are drawn within the range $[3, 14]$, the variation σ_i from $[0.4, 3.0]$, and the scale c_i from $[10, 100]$, while for $n(t)$ within $[1, 16]$, $[0.4, 3.0]$ and $[10, 100]$ respectively. We rescale the sum of these components to be normalized within a global maximum,

uniformly sampled from $[10, 100]$. Finally, we bias towards overloaded combinations of $\lambda(t)$ and $n(t)$ by requiring $\sum(\lambda(t) - n(t)) / \sum \lambda(t) > 0.3$. The latter bias filters out trajectories that are mostly overstaffed, leading to predominantly zero performance trajectories with low distributional training data contribution.

B.2. Simulation Procedures

For each simulation sample, we begin with an initially empty system and obtain the Monte Carlo estimated performance metric trajectories by averaging N_s independent simulation paths. We set $N_s = 10,000$, something that can be adjusted for lower scale systems, aiming for a standard error within 0.01 for every learned metric. For both simulations and exact computations we operate in a finite time interval $[0, T]$ with $T = 17$ and discretize time with a step size $\Delta t \equiv 0.05$.

Appendix C: Training Procedures and Hyperparameters

This appendix provides complete training details for the neural networks described in Section 3.2.4.

To achieve better network convergence, we train our two neural networks separately.

MQNet Training: For the MQNet network $\mathcal{F}^Q$, we use 10000 sampled trajectories of $\lambda(t)$ and $n(t)$ (generated using the input sampling strategy described in Appendix B), with a 20% test held-out breakdown. We use 30 epochs, a batch size of 5000 training samples, a learning rate of 10^{-4} , a hidden width of 5000 neurons, and a weight decay of 0.01. For the $\mathcal{S}^W$ network, the hyperparameters we use are 30, 2000, 10^{-4} , 2000, and 0.01 respectively. The total parameters for $\mathcal{F}^Q$ and $\mathcal{F}^W$ are approximately 150M and 24M respectively. In all model training, we backpropagate using the L1 loss function, and we use Adam optimization.

NMQT Training: For the NMQT networks, we employ a transfer learning approach using 100 sampled trajectories (1% of the MQNet training set) generated from non-Markovian simulations. We use 200 epochs with a batch size of 100 samples, a learning rate of 10^{-7} (10^3 times smaller than MQNet pre-training to preserve learned features), and a weight decay of 0.01. We experiment with three transfer learning strategies: (a) freezing the first 12 layers of MQNet and fine-tuning only the last 4 layers, (b) fine-tuning the entire network with the reduced learning rate compared to the MQNet's training learning rate, and (c) adding new NMQT-specific layers with 2000 hidden neurons while keeping MQNet frozen. The total parameters for the combined MQNet+NMQT architecture are approximately 180M.

For training the networks, especially during the fine-tuning stage with limited and potentially noisy simulation data, specific hyperparameter choices are beneficial. Using large batch sizes in conjunction with very small learning rates, combined with smooth non-linear activation functions, helps the networks mitigate noise and avoid overfitting, leading to smoother and more stable predictions.

Appendix D: Prediction Accuracy under Non-Markovian Distributions

This appendix provides additional details on the evaluation of QUANT's prediction accuracy under non-Markovian service and abandonment distributions, complementing the results reported in Section 4.2.

To test QUANT's robustness on NSNM queues with general service and abandonment distributions, we consider hyperexponential H_2 and Erlang (with 4 phases) E_4 with density functions:

$$g_{H_2}(x) = p\mu_1 e^{-\mu_1 x} + (1-p)\mu_2 e^{-\mu_2 x} \quad \text{and} \quad g_{E_4}(x) = \frac{(4\mu)^4 x^3 e^{-4\mu x}}{6},$$

with $p = 0.5(1 - \sqrt{0.6})$, $\mu_1 = 1 - \sqrt{0.6}$, and $\mu_2 = 1 + \sqrt{0.6}$ for H_2 , and $\mu = 1$ for E_4 . All three cases (M , H_2 and E_4) have equal means. To measure the distribution beyond the mean, we consider the *squared coefficient of variation* (SCV) c^2 : we set $c^2 = 4$ and $1/4$ for H_2 and E_4 respectively. This is to clearly separate from the base Markovian case where $c^2 = 1$ for exponential distribution.

Appendix E: Optimization Configuration Details

This appendix provides complete details of the experimental configuration for the optimization studies presented in Section 5.

E.1. Test Scenarios

We construct 200 distinct operational scenarios spanning horizons of $T = 17$ time units with $\Delta t = 0.05$ discretization, matching the temporal configuration from Section 3. The arrival patterns are generated using the basis function approach from Section 3.2.2, yielding diverse temporal profiles including single-peak, double-peak, multi-modal, and high-volatility patterns.

Service and abandonment distributions span three families matching those analyzed in Section 3: $M_t/M/n_t + M$ (Markovian, $SCV = 1$), $M_t/E_4/n_t + E_4$ (Erlang-4, $SCV = 0.25$), and $M_t/H_2/n_t + H_2$ (hyperexponential, $SCV = 4$). All systems maintain mean service time $1/\mu = 1$ and mean abandonment time $1/\theta = 1$. Complete distributional specifications are provided in Section 4.2.

E.2. Validation Methodology

To validate QUANT-OS effectiveness, we employ a two-stage verification process as described in Section 5.1. First, we run QUANT-OS given an arrival pattern $\{\lambda_t\}_{t=1}^T$, predetermined staffing $\{n_t\}_{t=1}^T$, and constraint policy $\Pi(\cdot)$ to obtain an optimized staffing schedule $\{n_t^*\}_{t=1}^T$ and QUANT's predicted performance trajectories.

Second, we simulate the actual $G_t/G/n_t + GI$ system using the optimized staffing $\{n_t^*\}_{t=1}^T$ with $N_s = 1,000$ independent replications, following the simulation procedures detailed in Appendix B. The number of replications is lower than the simulated training data of QUANT as we are less interested in random fluctuations of simulation in this validation. We declare a constraint satisfied if the simulated metric remains within the specified bounds for at least 95% of the time steps in the operational horizon $[0, T]$.

For scenarios with multiple constraints, we require all constraints to be simultaneously satisfied at each time step. The reported constraint violation rates in Section 5 represent the percentage of time steps where at least one constraint is violated in the simulated performance.

Appendix F: Alternative Optimization Methods

This appendix presents two alternative optimization methods for the staffing problem and provides a comparative discussion of their empirical performance. Algorithm 1 in Section 5.1 is referred to as the batch evaluation method. Here we describe two alternative optimization approaches that can be implemented by modifying the search procedure (lines 7-16 of Algorithm 1).

F.1. Binary Search Implementation

The binary search method exploits monotonicity (higher staffing yields better performance). We initialize with $n_{low} = \underline{n}$ and $n_{high} = \bar{n}$, then evaluate midpoint $n_{mid} = \lfloor (n_{low} + n_{high})/2 \rfloor$ at each iteration. If $\Pi(\cdot)$ is satisfied at n_{mid} , we update $n^* = n_{mid}$ and $n_{high} = n_{mid}$; otherwise $n_{low} = n_{mid}$. Termination at $n_{high} - n_{low} \leq 1$ guarantees $O(\log_2(\bar{n} - \underline{n}))$ convergence.

For multi-constraint problems, a staffing level is considered feasible only if $\Pi(\mathbf{S}_{t+1}) = \max_i \{\eta_i(t+1) - \bar{\eta}_i\} \leq 0$, ensuring all constraints are simultaneously satisfied.

Algorithm 2 Binary Search Optimization (replaces lines 7-16 of Algorithm 1)

```

1:  $n_{low} \leftarrow \underline{n}, n_{high} \leftarrow \bar{n}$ 
2: while  $n_{high} - n_{low} > 1$  do
3:    $n_{mid} \leftarrow \lfloor (n_{low} + n_{high})/2 \rfloor$ 
4:    $\bar{\mathbf{S}}_t \leftarrow [\mathbf{s}_t^\top \ \lambda_{t+1} \ n_{mid}] \in \mathbb{R}^{1 \times (d+2)}$ 
5:    $\mathbf{S}_{t+1}^o \leftarrow \mathcal{F}(\bar{\mathbf{S}}_t), \ \bar{\mathbf{S}}_{t+1}^o \in \mathbb{R}^{1 \times d}$ 
6:   if  $\Pi(\bar{\mathbf{S}}_{t+1}^o) \leq 0$  then
7:      $n_{t+1}^* \leftarrow n_{mid}, n_{high} \leftarrow n_{mid}$ 
8:   else
9:      $n_{low} \leftarrow n_{mid}$ 
10:  end if
11: end while

```

F.2. Gradient-Based Optimization Implementation

We consider the system evolution

$$S_{t+1} = \mathcal{F}(S_t, \beta_{t+1}),$$

where the control vector β_{t+1} includes the staffing decision n_{t+1} . Since the transition mapping $\mathcal{F}(\cdot)$ is implemented as a differentiable neural network surrogate, the following computational path is differentiable:

$$n_{t+1} \rightarrow \beta_{t+1} \rightarrow S_{t+1} \rightarrow \Pi(S_{t+1}).$$

As a result, the gradient of the penalty with respect to the staffing decision, $\nabla_{n_{t+1}} \Pi(S_{t+1})$, can be obtained via automatic differentiation (e.g., PyTorch's autograd).

We therefore optimize the surrogate objective

$$L(n_{t+1}) = \Pi(S_{t+1}) + \epsilon n_{t+1}, \tag{9}$$

using gradient-based updates on a continuous relaxation of n_{t+1} , followed by projection and rounding to obtain an integer staffing decision.

We initialize $\tilde{n} = n_{t+1}$ with gradient tracking enabled, compute the loss and gradient, and update:

$$\tilde{n} \leftarrow \max(\underline{n}, \min(\tilde{n} - \alpha \cdot \nabla_{\tilde{n}} L(\tilde{n}), \bar{n})), \tag{10}$$

where the min and max operations enforce the feasible range $[\underline{n}, \bar{n}]$.

The algorithm terminates when $|\nabla_{\tilde{n}} L(\tilde{n})| < c$ or after I_{max} iterations, then rounds to $n_{t+1}^* = \lfloor \tilde{n} + 0.5 \rfloor$. We use standard gradient descent with fixed learning rate $\alpha = 0.1$, projection onto $[\underline{n}, \bar{n}]$, and early stopping based on gradient magnitude.

Algorithm 3 Gradient-Based Optimization (replaces lines 7-16 of Algorithm 1)**Require:** Learning rate α , convergence threshold c , max iterations I_{max} , regularization weight ϵ

```

1:  $\tilde{n} \leftarrow n_{t+1}$  (enable gradient tracking),  $k \leftarrow 0$ 
2: while  $k < I_{max}$  do
3:    $\tilde{\mathbf{S}}_t \leftarrow [\mathbf{s}_t^\top \ \lambda_{t+1} \ \tilde{n}] \in \mathbb{R}^{1 \times (d+2)}$ 
4:    $\mathbf{S}_{t+1}^o \leftarrow \mathcal{F}(\tilde{\mathbf{S}}_t)$ ,  $\tilde{\mathbf{S}}_{t+1}^o \in \mathbb{R}^{1 \times d}$ 
5:    $L \leftarrow \Pi(\tilde{\mathbf{S}}_{t+1}^o) + \epsilon \cdot \tilde{n}$ 
6:   if  $|\nabla_{\tilde{n}} L(\tilde{n})| < c$  then break
7:   end if
8:    $\tilde{n} \leftarrow \max(1, \min(\tilde{n} - \alpha \cdot \nabla_{\tilde{n}} L(\tilde{n}), \tilde{n}))$ 
9:    $k \leftarrow k + 1$ 
10: end while
11:  $n_{t+1}^* \leftarrow \lfloor \tilde{n} + 0.5 \rfloor$ 

```

F.3. Algorithm Configuration

Table 6 provides the default parameter configuration used across all QUANT-OS experiments. These values are referenced in Algorithm 1 and discussed in Section 5.1.

Table 6 Default QUANT-OS algorithm parameters.

Parameter	Value	Description
$\underline{n}$	1	Minimum feasible staffing level
$\bar{n}$	100	Maximum feasible staffing level
c	0.01	Convergence threshold (gradient-based only)
α	0.1	Learning rate (gradient-based only)
I_{max}	50	Maximum iterations (gradient-based only)
ϵ	0.1	Regularization weight (gradient-based only)

Parameters apply to all three optimization methods unless noted otherwise.

For binary search, bisection continues until the search interval narrows to a single server. For batch evaluation, all candidates within $[\underline{n}, \bar{n}]$ are evaluated in a single vectorized forward pass. For gradient-based optimization, we employ gradient descent with early stopping when the gradient magnitude falls below c or after I_{max} iterations.

Our implementation uses PyTorch for automatic differentiation in the gradient-based method. For binary and batch methods, gradient tracking is disabled to reduce memory overhead. All methods use the same QUANT model checkpoint, ensuring consistent predictions across optimization approaches.

The constraint policy function $\Pi(\mathbf{S}_t)$ is implemented as a modular component, allowing easy extension to new constraint types or combinations.

F.4. Method Comparisons

Table 7 presents convergence characteristics and computational efficiency across optimization methods.

Table 7 QUANT-OS convergence and computational performance.

Method	Iterations per step	Computation time (sec)	Convergence success rate
Binary search	8.2 ± 0.5	1.2	100%
Batch evaluation	1.0 ± 0.0	0.8	100%
Gradient-based	34.5 ± 17.7	10.1	97.2%

Statistics across 200 test scenarios for horizon $T = 17$ (340 time steps).

Batch and binary methods achieve 100% convergence success, completing optimizations in 0.8 and 1.2 seconds respectively. Gradient-based optimization achieves 97.2% success in 10.1 seconds, with occasional failures attributable to suboptimal local minima, subject to hyperparameter tuning.

Appendix G: The $G_t/GI/n_t$ Queueing Systems without Customer Abandonment

This appendix demonstrates QUANT's capability to analyze queueing systems without customer abandonment, as mentioned in Section 1. This represents a key advantage over NeuraliNQ, which required abandonment due to its reliance on pointwise stationary approximation.

Figure 8 shows three queueing systems without customer abandonment, with the same distributional configuration as Figure 2. As expected, such systems experience higher congestion in understaffed situations. Nevertheless, QUANT's neural network structure learns these dynamics effectively, achieving similar accuracy ($\text{TAMSE}_{G_t/GI/n_t} = 0.06803$) as systems with abandonment.

Appendix H: MQSolver: Complete Algorithmic Details

This appendix provides the complete algorithmic details for MQSolver, the Markovian queue performance oracle introduced in Section 3.2.1. MQSolver computes exact transient performance metrics for the $M_t/M/n_t + M$ queueing system by exploiting its Markovian structure through uniformization techniques.

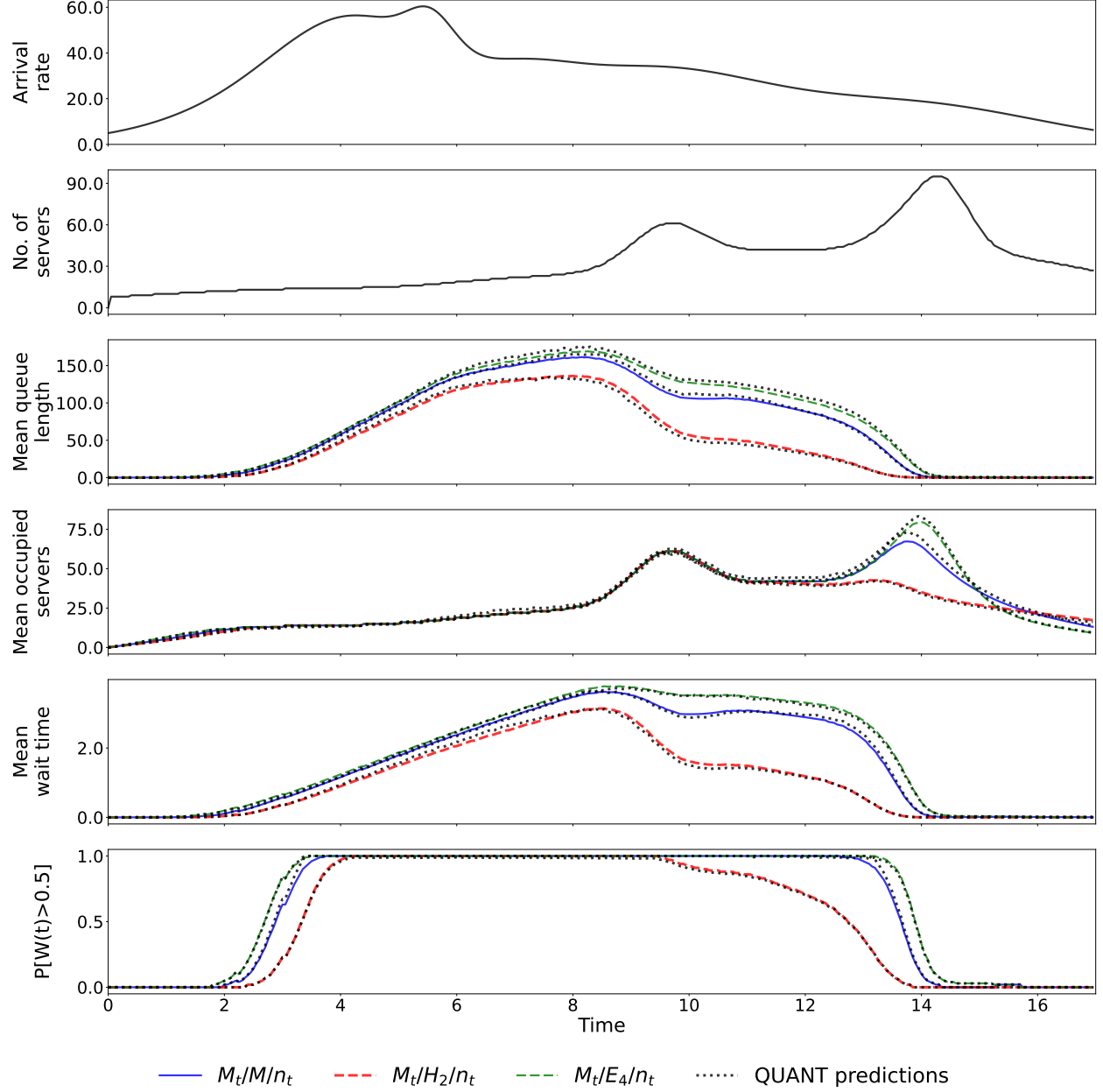
H.1. Queue-Length Distribution

We discretize the total time horizon T into M small successive intervals of length $h = T/M$. For a sufficiently small h , at most one event can occur in each interval, which corresponds to one of the following events: arrival of a new customer or departure of an existing customer (either due to service completion or abandonment).

To ensure computational feasibility, we truncate the state space at a maximum queue length N (a large number). At each discrete time step $t_k = k \cdot h$ for $k = 0, 1, \dots, M - 1$, the total queue length evolves according to a birth-and-death process with a finite state space $\mathcal{S} = \{0, 1, \dots, N\}$. When $X(t_k) = i$ for $i \in \mathcal{S}$, the system makes transitions with the following probabilities:

$$\begin{aligned}
 P_{i,j}(t_k) &\equiv \mathbb{P}(X(t_{k+1}) = j | X(t_k) = i) \\
 &= \begin{cases} \lambda(t_k)h, & j = i + 1 \quad (\text{An arrival}) \\ (n_{t_k} \wedge i)\mu h + (i - n_{t_k})^+ \theta h, & j = i - 1 \quad (\text{A departure}) \\ 1 - \lambda(t_k)h - (n_{t_k} \wedge i)\mu h - (i - n_{t_k})^+ \theta h, & j = i \quad (\text{No event}) \\ 0, & \text{otherwise,} \end{cases}
 \end{aligned}$$

where $a \wedge b \equiv \min(a, b)$ and $a^+ \equiv \max(a, 0)$.

Figure 8 QUANT predictions of Markovian and non-Markovian systems without customer abandonment.

We denote $\pi(t_k) \equiv [\pi_0(t_k), \pi_1(t_k), \dots, \pi_N(t_k)]$ as the transient distribution of $X(t_k)$, that is, $\pi_i(t_k) \equiv \mathbb{P}(X(t_k) = i)$. Starting in a given initial distribution $\pi(0)$ (e.g., $\pi(0) = [1, 0, \dots, 0]$ indicating an empty system), the transient distribution at all time steps can be obtained via matrix multiplications:

$$\pi(t_k) = \pi(0)\mathbf{P}(t_0)\mathbf{P}(t_1) \cdots \mathbf{P}(t_{k-1}), \quad (11)$$

where $\mathbf{P}(t_k)$ is the one-step transition probability matrix with its $(i, j)^{\text{th}}$ element being $P_{i,j}(t_k)$.

Once we have the queue-length distribution, we can immediately compute the queue-based service-level metrics such as the mean total queue length $\mathbb{E}[X(t_k)]$, mean length of the waiting queue $\mathbb{E}[Q(t_k)]$, the server utilization

$\mathbb{E}[B(t_k)]/n_{t_k}$ (the ratio of mean number of busy servers $B(t_k)$ to the total number of servers), and rates of abandonment $\alpha(t_k)$ and service completion $\sigma(t_k)$, at all time steps t_k , for $k = 0, \dots, M-1$:

$$\begin{aligned} \mathbb{E}[X(t_k)] &= \sum_{i=0}^N i \cdot \pi_i(t_k), \quad \mathbb{E}[Q(t_k)] = \sum_{i=0}^N (i - n_{t_k})^+ \cdot \pi_i(t_k), \\ \frac{\mathbb{E}[B(t_k)]}{n_{t_k}} &= \frac{1}{n_{t_k}} \sum_{i=0}^N (i \wedge n_{t_k}) \cdot \pi_i(t_k), \quad \alpha(t_k) = \theta \mathbb{E}[Q(t_k)], \quad \sigma(t_k) = \mu \mathbb{E}[B(t_k)]. \end{aligned} \quad (12)$$

H.2. Waiting Times Conditional on the Queue Length

To compute the waiting time distribution, we first condition on the total number of customers in the system at arrival time. We first treat the potential waiting time (PWT) $W(t)$, the offered waiting time for a customer arrival at t (if any) assuming the customer is infinitely patient.

If $x < n_{t_k}$, we have $W(t_k) = 0$. If $x \geq n_{t_k}$, then all servers are busy and there are $q = x - n_{t_k} \geq 0$ customers waiting in the queue. Let $W_q(t_k)$ be the waiting time of a customer arrival (which we call the “tagged customer”) conditional on q waiting customers at t_k .

We model $W_q(t_k)$ by a phase-type distribution, which is the time for a pure death process to reach an absorbing state 0, beginning at an initial state $q+1$. Specifically, the state space is $\mathcal{S}_{t_k, q} \equiv \{0, 1, \dots, q+1\}$ where $j \in \mathcal{S}_{t_k, q}$ tracks the customer’s queueing position after time t_k , with $q+1$ being the initial position at t_k and 0 meaning the customer first entering service at the future time $t_k + W_q(t_k)$. Here, a “death” corresponds to a departure from the waiting line due to abandonment or a service completion from n_{t_k} busy servers.

We again consider a discrete-time framework. After a sufficiently small time step h , the tagged customer’s queue position decreases by 1 (i.e., moving towards the head of the line) with probability $(n_{t_k}\mu + (j-1)\theta)h$ when at position j , while its queueing position remains unchanged with probability $1 - (n_{t_k}\mu + (j-1)\theta)h$. Hence, for row j where $1 \leq j \leq q+1$, the transition probability to column $j-1$ is $(n_{t_k}\mu + (j-1)\theta)h$ and to column j is $1 - (n_{t_k}\mu + (j-1)\theta)h$, with all other entries being zero. The transition probability matrix is:

$$\begin{array}{c|cccccc} & 0 & 1 & 2 & \cdots & q & q+1 \\ \hline 0 & 1 & 0 & 0 & \cdots & 0 & 0 \\ 1 & n_{t_k}\mu h & 1 - n_{t_k}\mu h & 0 & \cdots & 0 & 0 \\ 2 & 0 & (n_{t_k}\mu + \theta)h & 1 - (n_{t_k}\mu + \theta)h & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ q & 0 & 0 & 0 & \cdots & 1 - (n_{t_k}\mu + (q-1)\theta)h & 0 \\ q+1 & 0 & 0 & 0 & \cdots & (n_{t_k}\mu + q\theta)h & 1 - (n_{t_k}\mu + q\theta)h \end{array} \quad (13)$$

Let $\mathbf{P}_{t_k}(q)$ denote the $(q+1) \times (q+1)$ submatrix excluding the 0th row and column. When the staffing level changes between time steps, we must also account for instantaneous repositioning. If staffing increases from $n_{t'_{i-1}}$ to $n_{t'_i}$ by $\Delta n = n_{t'_i} - n_{t'_{i-1}} > 0$, then Δn customers at the head of the queue immediately enter service, causing each remaining customer’s position to decrease by Δn . We capture this through a repositioning matrix $\mathbf{R}_i(\Delta n)$ where $R_{j,k} = 1$ if $k = (j - \Delta n)^+$ and $R_{j,k} = 0$ otherwise. For staffing unchanged or decreased, $\mathbf{R}_i = \mathbf{I}$ (the identity matrix).

For $\tau > 0$, $q \geq 0$, and $t'_i \equiv t_k + ih$, $M_\tau \equiv \lceil \tau/h \rceil$, the conditional waiting time distribution given $X(t_k) = x$ is:

$$\mathbb{P}(W(t_k) \leq \tau \mid X(t_k) = x) = \begin{cases} 1, & x < n_{t_k}, \\ 1 - \boldsymbol{\beta}_x \prod_{i=1}^{M_\tau} [\mathbf{R}_i \cdot \mathbf{P}_{t'_i}(x - n_{t_k})] \mathbf{e}, & x \geq n_{t_k}, \end{cases} \quad (14)$$

where $\beta_x \equiv [0, \dots, 0, 1]$ is a row vector with q zeros and a 1 in its last cell, and $\mathbf{e} \equiv [1, \dots, 1]^T$ is a column vector of all ones.

Using the queue-length distribution derived in Section H.1, we retrieve the unconditional waiting-time distribution:

$$\mathbb{P}(W(t_k) \leq \tau) = \sum_{x=0}^N \mathbb{P}(W(t_k) \leq \tau \mid X(t_k) = x) \pi_x(t_k), \quad (15)$$

from which the tail probability of delay $\mathbb{P}(W(t_k) > \tau) = 1 - \mathbb{P}(W(t_k) \leq \tau)$ follows immediately.

H.3. Mean Waiting Time via Backward Recursion

The mean waiting time $\mathbb{E}[W(t_k)]$ can be obtained using the waiting-time distribution in (15) at different τ . We hereby give a more efficient way to compute $\mathbb{E}[W(t_k)]$ using backward recursion.

For $t_k = kh$, we define $\mathbb{E}[W(t_k, q)]$ as the expected remaining waiting time at time t_k for a customer who has a queueing position $q > 0$ in the waiting line. Let $\Delta n_k = (n_{t_{k+1}} - n_{t_k})^+$ denote the staffing increase (zero if unchanged or decreased). The recursion depends on whether the customer enters service due to the staffing change.

If $q \leq \Delta n_k$, the customer immediately enters service and $\mathbb{E}[W(t_k, q)] = h$. Otherwise, let $q' = q - \Delta n_k$ denote the new position (with $q' = q$ when $\Delta n_k = 0$). The recursion is:

$$\mathbb{E}[W(t_k, q)] = h + (n_{t_{k+1}}\mu + (q' - 1)\theta)h \cdot \mathbb{E}[W(t_{k+1}, q' - 1)] + (1 - (n_{t_{k+1}}\mu + (q' - 1)\theta)h) \cdot \mathbb{E}[W(t_{k+1}, q')], \quad (16)$$

where the first term accounts for advancing one position (service completion or abandonment of a customer ahead) and the second term accounts for remaining in position. The recursion implicitly assumes $\mathbb{E}[W(t_k, 0)] = 0$ for all t_k .

At $q = 1$ with no staffing increase ($\Delta n_k = 0$), the boundary condition is:

$$\mathbb{E}[W(t_k, 1)] = h + (1 - n_{t_{k+1}}\mu h) \cdot \mathbb{E}[W(t_{k+1}, 1)], \quad (17)$$

since the tagged customer only needs to wait for a service completion. If $\Delta n_k \geq 1$, then $\mathbb{E}[W(t_k, 1)] = h$.

At the end of the time horizon, assuming the system reaches a stationary regime with constant staffing n_T (i.e., $\mathbb{E}[W(T, q)] = \mathbb{E}[W(T + h, q)]$), we have:

$$\mathbb{E}[W(T, q + 1)] = \frac{1}{n_T\mu + q\theta} + \mathbb{E}[W(T, q)], \quad (18)$$

with base case $\mathbb{E}[W(T, 1)] = \frac{1}{n_T\mu}$. The unconditional mean waiting time is:

$$\mathbb{E}[W(t_k)] = \sum_{x=n_{t_k}}^N \mathbb{E}[W(t_k, x - n_{t_k} + 1)] \cdot \pi_x(t_k). \quad (19)$$

H.4. Actual Waiting Time (AWT)

The above analysis can be extended to the actual waiting time (AWT) $W_a(t) \equiv \min(W(t), A)$, with A denoting a generic abandonment time. Unlike PWT, AWT accounts for the possibility that the tagged customer abandons before reaching service.

For the distribution of AWT, we modify the transition probability matrix (13) to include the tagged customer's own abandonment. For row j where $1 \leq j \leq q + 1$, the non-zero entries are: column 0 has θh (tagged customer abandons),

column $j - 1$ has $(n_{t_k}\mu + (j - 1)\theta)h$ (service completion or abandonment of a customer ahead), and column j has $1 - (n_{t_k}\mu + j\theta)h$ (no event). For row 1, the entries for columns 0 and 1 are $(n_{t_k}\mu + \theta)h$ and $1 - (n_{t_k}\mu + \theta)h$ respectively.

$$\begin{array}{c|cccccc}
 & 0 & 1 & 2 & \cdots & q & q+1 \\
 0 & 1 & 0 & 0 & \cdots & 0 & 0 \\
 1 & (n_{t_k}\mu + \theta)h & 1 - (n_{t_k}\mu + \theta)h & 0 & \cdots & 0 & 0 \\
 2 & \theta h & (n_{t_k}\mu + \theta)h & 1 - (n_{t_k}\mu + 2\theta)h & \cdots & 0 & 0 \\
 \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\
 q & \theta h & 0 & \cdots & 1 - (n_{t_k}\mu + q\theta)h & 0 & \\
 q+1 & \theta h & 0 & 0 & \cdots & (n_{t_k}\mu + q\theta)h & 1 - (n_{t_k}\mu + (q+1)\theta)h
 \end{array} \quad (20)$$

The conditional distribution for AWT can be computed using (14) with the submatrix from (20).

For the mean AWT, the recursion follows the same structure as PWT. If $q \leq \Delta n_k$, then $\mathbb{E}[W_a(t_k, q)] = h$. Otherwise, with $q' = q - \Delta n_k$:

$$\mathbb{E}[W_a(t_k, q)] = h + (n_{t_{k+1}}\mu + (q' - 1)\theta)h \cdot \mathbb{E}[W_a(t_{k+1}, q' - 1)] + (1 - (n_{t_{k+1}}\mu + q'\theta)h) \cdot \mathbb{E}[W_a(t_{k+1}, q')], \quad (21)$$

where the “stay” probability includes the tagged customer’s own abandonment rate.

At $q = 1$ with no staffing increase, $\mathbb{E}[W_a(t_k, 1)] = h + (1 - (n_{t_{k+1}}\mu + \theta)h) \cdot \mathbb{E}[W_a(t_{k+1}, 1)]$. If $\Delta n_k \geq 1$, then $\mathbb{E}[W_a(t_k, 1)] = h$.

At the end of the time horizon under stationarity:

$$\mathbb{E}[W_a(T, q+1)] = \frac{1}{n_T\mu + (q+1)\theta} + \frac{n_T\mu + q\theta}{n_T\mu + (q+1)\theta} \cdot \mathbb{E}[W_a(T, q)], \quad (22)$$

with base case $\mathbb{E}[W_a(T, 1)] = \frac{1}{n_T\mu + \theta}$.